



## ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

В настоящей пояснительной записке применяются следующие сокращения, термины и определения:

API – программный интерфейс приложения;  
APK – формат установочного пакета Android-приложения;  
CRUD – набор операций создания, чтения, обновления и удаления данных;  
GPS – глобальная система позиционирования;  
HTTP – протокол передачи гипертекста;  
HTTPS – защищённый протокол передачи гипертекста;  
JSON – текстовый формат обмена структурированными данными;  
MVC – архитектурный шаблон «модель – представление – контроллер»;  
REST – архитектурный стиль построения веб-API;  
UI – пользовательский интерфейс;  
UML – унифицированный язык моделирования;  
БД – база данных;  
КВ – капитальные вложения;  
НДС – налог на добавленную стоимость;  
СУБД – система управления базами данных;  
ФЗП – фонд заработной платы;  
ФРВ – фонд рабочего времени;  
ФСЗН – Фонд социальной защиты населения.

## ВВЕДЕНИЕ

Каждый житель города хотя бы раз сталкивался с ситуацией, когда во дворе долго не убирают мусор, урна возле остановки переполнена, рядом с контейнерной площадкой образовалась свалка, а сообщить об этом быстро и удобно некуда. Такие проблемы кажутся бытовыми, но именно из них складывается общее состояние городской среды. Если обращения жителей теряются, долго рассматриваются или не доходят до ответственных служб, загрязнения остаются нерешёнными, а доверие к городскому управлению снижается.

Чистота улиц, дворов, парков и общественных пространств напрямую влияет на комфорт проживания, санитарное состояние территории и внешний облик города. При этом для эффективного решения подобных проблем важно не только принять обращение от жителя, но и точно зафиксировать место загрязнения, получить фотографию, назначить ответственного исполнителя, проконтролировать выполнение работы и сохранить историю обработки заявки.

На практике процесс передачи информации о загрязнениях часто организован недостаточно удобно. Жители могут обращаться по телефону, через социальные сети, мессенджеры или лично в соответствующие организации. Однако такие способы не всегда позволяют указать точные координаты проблемы, приложить фото, отследить статус обращения и получить понятную обратную связь. В результате часть заявок обрабатывается медленно, а администрации и службам уборки сложнее контролировать объём работ и качество их выполнения.

Для решения данной проблемы разрабатывается информационная система мониторинга и обработки заявок о загрязнениях городской среды «Чистый город». Система предназначена для трёх основных групп пользователей: жителей, служб уборки и администрации. Жители получают возможность быстро отправить заявку с фотографией и геолокацией, исполнители — просматривать назначенные задачи, менять статусы и подтверждать выполнение работ, а администрация — управлять заявками, исполнителями, справочниками и аналитикой. Такие функции предусмотрены в техническом задании проекта, где также указаны Android-приложение, веб-панель администратора, backend API, база данных, файловое хранилище и средства работы с картами.

Главное преимущество системы заключается в том, что она объединяет весь процесс обработки обращения в единую цепочку: от фиксации проблемы жителем до закрытия заявки исполнителем. Пользователь не просто отправляет

сообщение, а может видеть дальнейшее изменение статуса. Исполнитель получает конкретную задачу с местоположением и фотографией, что упрощает выезд на место. Администратор, в свою очередь, получает инструмент контроля, распределения заявок и анализа ситуации по категориям, районам, срокам и исполнителям.

# 1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

## Описания предметной области

Современные города сталкиваются с серьёзной проблемой оперативного реагирования на нарушения санитарного состояния городской среды: несанкционированные свалки, переполненные мусорные урны, загрязнение дворовых территорий и общественных пространств. Традиционные механизмы взаимодействия граждан с коммунальными службами — телефонные звонки, обращения через МФЦ или бумажные жалобы — обладают рядом существенных недостатков: высокая трудоёмкость обработки обращений, отсутствие прозрачности для заявителя, потеря информации при передаче между отделами, невозможность контроля исполнения в реальном времени.

Предметной областью данной дипломной работы является процесс управления заявками о загрязнении городской среды — от момента обнаружения проблемы гражданином до подтверждения её устранения коммунальной службой.

Жизненный цикл заявки охватывает следующие этапы: фиксация проблемы гражданином с прикреплением фотографии и геолокации, проверка и назначение исполнителя администратором, принятие заявки в работу исполнителем → выполнение работ и закрытие заявки с фотоподтверждением, уведомление заявителя об устранении проблемы.

Важным аспектом предметной области является территориальная привязка заявок. Каждое обращение имеет географические координаты, что позволяет визуализировать проблемные зоны на карте, быстрее передавать информацию ответственным службам и анализировать распределение обращений по территории города.

Анализ существующих решений в данной области показывает, что белорусские электронные сервисы, ЖКХ-порталы и городские системы приёма обращений в основном ориентированы на регистрацию заявок и информирование граждан. При этом дальнейшая работа с обращениями часто требует дополнительных организационных действий: распределения заявок между исполнителями, контроля их выполнения, планирования выездов и подготовки отчётности. Не все существующие решения позволяют объединить эти процессы в единой системе, что делает актуальной разработку комплексной информационной системы.

## 1.2 Постановка задачи

Целью данного дипломного проекта является проектирование и разработка информационной системы мониторинга и обработки заявок о загрязнении городской среды «Чистый город», обеспечивающей полный цикл работы с обращениями граждан — от момента фиксации проблемы до подтверждения её устранения.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Анализ предметной области и существующих решений — изучить действующие механизмы взаимодействия граждан с коммунальными службами, выявить их недостатки, провести обзор аналогичных программных продуктов и обосновать необходимость разработки собственной системы.

2. Проектирование архитектуры системы — определить состав компонентов, разработать структуру базы данных, спроектировать REST API, описать роли пользователей и жизненный цикл заявки.

3. Разработка мобильного Android-приложения — реализовать два режима работы: режим жителя, позволяющий подавать заявки с фотографией и геолокацией, и режим исполнителя, обеспечивающий приём задач, управление статусами и построение маршрута.

4. Разработка веб-панели администратора — реализовать интерфейс управления заявками, справочниками, исполнителями и зонами ответственности, а также инструменты аналитики и формирования отчётов.

5. Разработка серверной части — реализовать единый программный интерфейс для мобильного приложения и веб-панели, обеспечить аутентификацию, авторизацию по ролям и хранение фотоматериалов.

6. Тестирование программного продукта — провести проверку ключевых сценариев работы системы, включая позитивные и негативные тест-кейсы; выполнить функциональное тестирование серверной части с использованием PHPUnit.

Разрабатываемая система должна соответствовать следующим основным требованиям:

- поддержка трёх ролей пользователей: житель, исполнитель, администратор — с разграниченными правами доступа;
- обязательная геолокация и фотофиксация при создании заявки;
- хранение полной истории изменения статусов каждой заявки;

- отображение заявок на интерактивной карте с фильтрацией и кластеризацией;
- отображение изменений статуса заявки для жителя;
- защита данных посредством HTTPS, токеной API-аутентификации и разграничения доступа по ролям.

Результатом выполнения дипломного проекта должен стать готовый программный продукт, включающий мобильное приложение, веб-панель администратора и серверную часть, пригодный к развёртыванию и практическому использованию в условиях реального городского хозяйства.

### **Выбор и обоснование средств разработки**

Выбор инструментов и технологий разработки является одним из ключевых этапов проектирования информационной системы, поскольку от принятых решений напрямую зависят производительность, масштабируемость, удобство сопровождения и сроки реализации проекта. При выборе средств разработки для системы «Чистый город» учитывались следующие критерии: [13]

- соответствие технологии решаемым задачам и архитектуре системы;
- распространённость и наличие актуальной документации;
- открытая экосистема и доступность готовых библиотек;
- открытость инструментов;

Разрабатываемая система состоит из трёх взаимосвязанных компонентов, для каждого из которых были выбраны соответствующие технологии.

Мобильное приложение (Android) разрабатывается с использованием среды Android Studio и языка программирования Java. Android Studio является официальной IDE от Google для разработки Android-приложений и предоставляет визуальный редактор интерфейсов, эмулятор, инструменты профилирования и отладки [1, 4].

Серверная часть реализуется на PHP с использованием фреймворка Laravel. Laravel обеспечивает маршрутизацию HTTP-запросов, работу с контроллерами, валидацию входных данных, авторизацию пользователей, взаимодействие с базой данных через Eloquent ORM и хранение загружаемых файлов [5, 9].

Веб-панель администратора разработана средствами Laravel Blade, CSS и JavaScript. Для сборки фронтенд-ресурсов используется Vite, для аналитических графиков — Chart.js, а для отображения заявок на карте — Leaflet [6, 8, 9].

База данных. В качестве системы управления базами данных выбрана MySQL — распространённая реляционная СУБД с открытым исходным кодом, отличающаяся стабильностью, высокой скоростью работы и удобством

интеграции с веб-приложениями. MySQL подходит для хранения данных о пользователях, заявках, ролях, статусах выполнения, адресах и другой информации, необходимой для работы системы. Использование данной СУБД позволяет обеспечить централизованное хранение данных, их структурированную обработку и надёжное взаимодействие между мобильным приложением, серверной частью и веб-панелью администратора [11].

**Хранение данных.** В разработанной системе отдельное файловое хранилище не используется, поскольку основная работа приложения связана с обработкой структурированных данных заявок. Информация о пользователях, обращениях, адресах, статусах и исполнителях сохраняется в базе данных MySQL. Это упрощает архитектуру проекта и позволяет сосредоточиться на ключевых функциях системы: регистрации заявок, их распределении, контроле выполнения и отображении информации в административной панели.

**Карты и геолокация.** В мобильном приложении координаты определяются стандартными средствами Android LocationManager, а ручной выбор точки выполняется через WebView с картой Leaflet/OpenStreetMap. В административной веб-панели карта заявок также реализована на Leaflet с поддержкой маркеров и кластеризации [3, 7].

Совокупность выбранных технологий образует хорошо зарекомендовавший себя стек разработки, позволяющий реализовать все функциональные требования системы в установленные сроки.

### **основание выбора языка программирования**

В рамках разрабатываемой системы «Чистый город» используется несколько языков программирования, каждый из которых применяется в соответствующем компоненте системы. Выбор каждого языка обусловлен спецификой решаемых задач и особенностями целевой платформы.

Для реализации Android-приложения был выбран язык программирования Java. Такой выбор соответствует фактической структуре мобильного проекта CleanCity2, где основные экраны, модели и сетевой слой реализованы в Java-классах.

Java на протяжении многих лет является одним из основных языков Android-разработки и обладает большой базой документации, примеров и готовых библиотек. Для данного проекта это важно, так как приложение использует стандартные Android-компоненты, Retrofit, OkHttp и Material Components [1, 4].



– К преимуществам Java применительно к задачам данного проекта относятся:

– совместимость с Android SDK и библиотеками Material Components, Retrofit, OkHttp и Gson;

– стабильность языка и понятная объектно-ориентированная модель, удобная для описания экранов, моделей данных и API-клиента;

– широкая библиотечная база и наличие подробной документации, что ускоряет реализацию авторизации, сетевого обмена и загрузки фотографий;

– простота сопровождения проекта, так как логика экранов, обработчики событий и модели данных находятся в отдельных Java-классах.

PHP и Laravel — язык и фреймворк серверной части

Для разработки Backend API и веб-панели администратора используется PHP с фреймворком Laravel. Такой стек подходит для проекта, поскольку объединяет маршрутизацию, контроллеры, работу с базой данных, шаблоны веб-интерфейса и встроенные средства тестирования в одной экосистеме [5, 9].

Laravel предоставляет ряд преимуществ для проекта с несколькими ролями пользователей и сложной доменной моделью: [9]

- архитектура MVC – разделение моделей, представлений и контроллеров упрощает сопровождение кода;

- Eloquent ORM – удобная работа с таблицами MySQL, связями между пользователями, заявками, организациями, фотографиями и историей статусов;

- Blade и Vite – формирование административного интерфейса и подключение CSS/JavaScript-ресурсов без отдельного SPA-клиента;

- единая серверная платформа – REST API для мобильного приложения и веб-панель администратора используют общие модели, правила доступа и бизнес-логику.

### **основание выбора СУБД**

База данных является центральным компонентом информационной системы «Чистый город», обеспечивающим хранение и обработку всех данных системы: заявок граждан, пользователей, фотоматериалов, географических зон, истории статусов и аналитической информации. Выбор системы управления базами данных оказывает непосредственное влияние на производительность, надёжность и функциональные возможности всего программного продукта [11].

MySQL была выбрана в качестве основной СУБД для данного проекта по следующим причинам:

- MySQL является одной из наиболее популярных реляционных СУБД в мире, используется в миллионах проектов и обладает обширной документацией и активным сообществом разработчиков; [11]
- Высокая производительность – MySQL демонстрирует высокую скорость выполнения операций чтения и записи, что обеспечивает быструю обработку запросов при активной работе пользователей с системой;
- Полная поддержка реляционной модели – MySQL поддерживает внешние ключи, транзакции, индексы и хранимые процедуры, обеспечивая целостность данных при параллельной работе множества пользователей; [11]
- Веб-интерфейс phpMyAdmin предоставляет удобные графические инструменты для управления базой данных: создание и редактирование таблиц, выполнение SQL-запросов, импорт и экспорт данных, мониторинг состояния базы – без необходимости работы с командной строкой; [12]
- Лёгкость развёртывания – MySQL легко интегрируется со стеком PHP/Laravel посредством Eloquent ORM и миграций, что упрощает подключение серверной части к базе данных;
- Открытый исходный код – MySQL Community Edition распространяется бесплатно под лицензией GPL, что исключает лицензионные издержки при разработке и развёртывании системы;
- Поддержка пространственных типов данных – MySQL поддерживает типы данных POINT и пространственные функции ST\_Distance, ST\_Contains, достаточные для реализации геолокационных запросов в рамках данного проекта

## 2 ПРАКТИЧЕСКИЙ РАЗДЕЛ

### 2.1 Проектирование структуры базы данных

Проектирование структуры базы данных является важнейшим этапом разработки информационной системы, поскольку от корректности и полноты модели данных зависит работоспособность всех функциональных компонентов системы. База данных системы «Чистый город» реализована средствами СУБД MySQL и включает набор взаимосвязанных таблиц, отражающих доменную модель предметной области [11, 13].

При проектировании структуры базы данных соблюдались следующие принципы:

- нормализация до третьей нормальной формы (3НФ) для устранения избыточности данных; [13]
- обеспечение ссылочной целостности посредством внешних ключей;
- фиксация временных меток создания и обновления записей для ключевых таблиц.

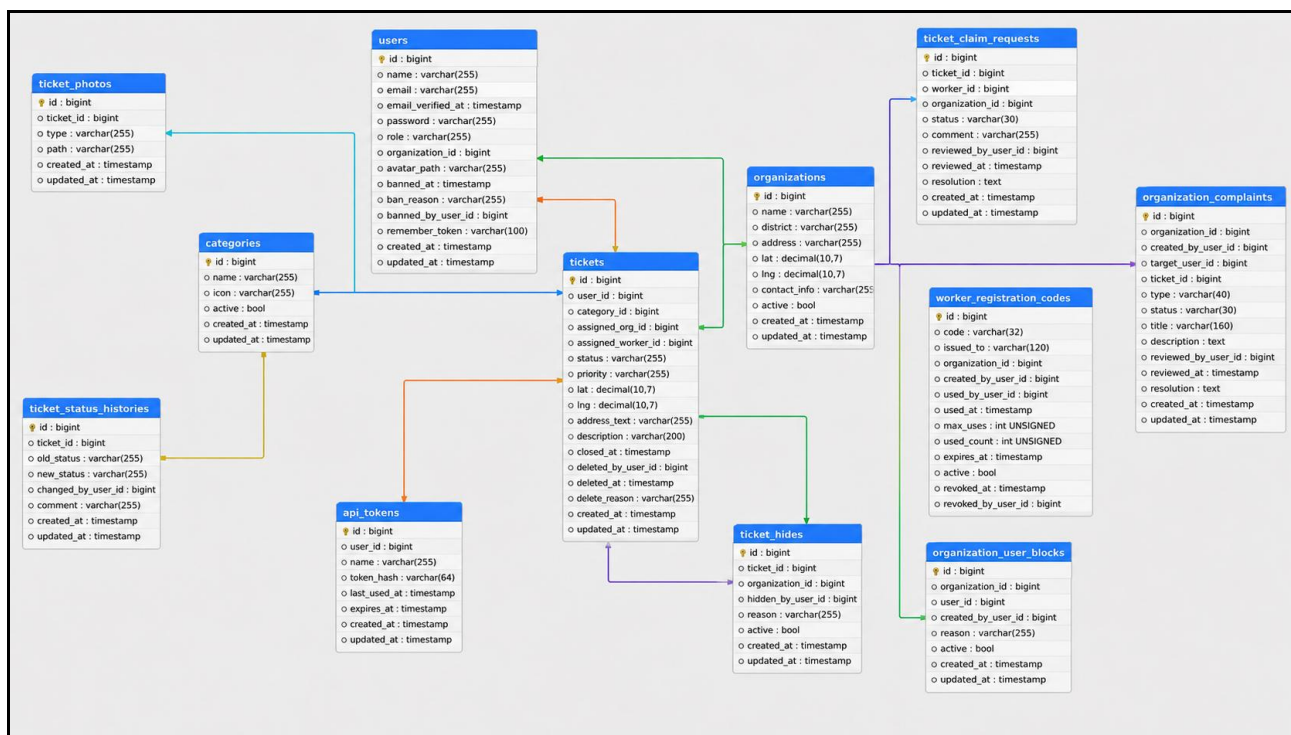


Рисунок 2.1 - Схема базы данных

В базе данных информационной системы «Чистый город» были выделены основные таблицы, каждая из которых отвечает за хранение определённого набора данных.

Таблица `users` предназначена для хранения информации о пользователях системы. В ней содержатся данные для авторизации, роль пользователя, сведения о привязке к организации, а также информация о блокировке пользователя при нарушении правил работы с системой.

Таблица `organizations` хранит сведения об организациях-исполнителях, которые занимаются обработкой заявок. В таблице указываются название организации, район обслуживания, адрес, координаты, контактная информация и признак активности.

Таблица `categories` используется для хранения категорий заявок. С её помощью заявки разделяются по типам проблем, например мусор, переполненная урна, несанкционированная свалка, граффити и другие виды загрязнений.

Таблица `tickets` является основной таблицей базы данных. В ней хранятся заявки, созданные пользователями. Каждая заявка содержит сведения о пользователе, категории проблемы, назначенной организации, исполнителе, статусе, приоритете, координатах, адресе, описании, дате создания и дате закрытия.

Таблица `ticket_photos` предназначена для хранения фотографий, прикрепленных к заявкам. В ней сохраняется путь к изображению, тип фотографии, а также идентификатор заявки, к которой относится данное изображение. Фотографии используются для подтверждения проблемы и результата выполненных работ.

Таблица `ticket_status_histories` хранит историю изменения статусов заявок. В ней фиксируется предыдущий статус, новый статус, пользователь, выполнивший изменение, комментарий и дата изменения. Это позволяет отслеживать полный жизненный цикл заявки.

Таблица `api_tokens` используется для хранения токенов доступа пользователей. Она необходима для авторизации пользователей при работе с мобильным приложением и серверной частью системы.

Таблица `worker_registration_codes` предназначена для хранения регистрационных кодов исполнителей. Такие коды позволяют привязать работника к определённой организации и ограничить регистрацию исполнителей только разрешёнными пользователями.

Таблица `ticket_claim_requests` хранит запросы исполнителей на получение заявки в работу. В ней фиксируется заявка, исполнитель, организация, статус запроса, комментарий и результат рассмотрения.

Таблица `ticket_hides` используется для хранения информации о скрытых заявках. Она позволяет скрывать отдельные заявки для организации или пользователя с указанием причины.

Таблица `organization_complaints` предназначена для хранения жалоб, связанных с организациями, пользователями или заявками. В ней указываются тип жалобы, описание, статус рассмотрения и результат принятого решения.

Таблица `organization_user_blocks` хранит сведения о блокировках пользователей в рамках конкретных организаций. Это позволяет ограничивать доступ отдельных пользователей к взаимодействию с организацией при наличии нарушений.

Между таблицами были установлены соответствующие связи:

- Один ко многим: один пользователь может создать множество заявок.
- Один ко многим: одна категория может относиться ко множеству заявок.
- Один ко многим: одна организация может быть назначена для выполнения множества заявок.
- Один ко многим: один исполнитель может быть ответственным за несколько заявок.
- Один ко многим: одна заявка может иметь несколько прикрепленных фотографий.

Для обеспечения целостности данных были предусмотрены связи между таблицами по идентификаторам. Основные связи строятся через поля `user_id`, `category_id`, `ticket_id`, `organization_id`, `assigned_org_id`, `assigned_worker_id` и другие внешние идентификаторы. Такой подход позволяет связать заявку с пользователем, категорией, организацией, исполнителем, фотографиями и историей изменения статусов.

Первичные ключи были заданы для основных таблиц с использованием идентификатора `id`, что обеспечивает уникальность каждой записи и упрощает управление данными. Внешние ключи позволяют избежать логических ошибок при добавлении, изменении и удалении информации, так как каждая связанная запись относится к существующей сущности базы данных.

Проектирование базы данных выполнялось с учётом требований к удобству хранения информации, расширяемости и дальнейшему развитию системы.

## 2.2 Проектирование и разработки приложения

Проектирование и разработка приложения информационной системы «Чистый город» выполнялись с учётом требований, предъявляемых к системе мониторинга и обработки заявок о загрязнении городской среды. Основной задачей разрабатываемого программного продукта является обеспечение удобного взаимодействия между жителями города, исполнителями и администратором системы.

Система предназначена для автоматизации процесса создания, обработки и контроля заявок. Житель должен иметь возможность быстро сообщить о проблеме, прикрепить фотографию и передать координаты места загрязнения. Исполнитель должен получать назначенные заявки, изменять их статусы и подтверждать выполнение работ фотографией. Администратор, в свою очередь, должен контролировать весь процесс, назначать исполнителей, управлять справочниками и просматривать информацию по заявкам. Данные функции соответствуют требованиям технического задания, в котором система разделяется на Android-приложение, веб-панель администратора, backend API, базу данных и файловое хранилище [10, 13].

Разрабатываемое приложение построено по клиент-серверной архитектуре. Такая архитектура позволяет разделить пользовательский интерфейс, бизнес-логику и хранение данных. Клиентская часть отвечает за взаимодействие пользователя с системой, отображение данных, ввод информации и отправку запросов. Серверная часть обрабатывает поступающие запросы, выполняет проверку данных, управляет заявками и обращается к базе данных. База данных используется для хранения пользователей, организаций, категорий, заявок, фотографий, статусов и другой служебной информации.

Общая структура системы включает следующие компоненты:

- мобильное приложение;
- административную часть для управления заявками;
- серверную часть, реализующую обработку запросов;
- базу данных для хранения информации;

Мобильное приложение для жителя предназначено для создания заявок о загрязнении городской среды. После авторизации пользователь получает доступ к основным функциям системы. На главном экране предусмотрена возможность создания новой заявки. Для этого пользователь выбирает категорию проблемы, добавляет фотографию, при необходимости указывает комментарий и отправляет обращение. Место загрязнения может быть определено автоматически с

помощью геолокации устройства либо выбрано пользователем вручную на карте. После отправки заявка сохраняется в базе данных и становится доступной для дальнейшей обработки.

Отдельное внимание при разработке уделяется удобству создания заявки. Процесс должен занимать минимальное количество действий, так как основная цель приложения – быстро зафиксировать проблему. Пользователь не должен заполнять большое количество полей. Основными обязательными данными являются категория, фотография и местоположение. Дополнительный комментарий используется для уточнения информации о проблеме [10].

Интерфейс исполнителя предназначен для работы с назначенными заявками. Исполнитель после входа в систему видит список обращений, которые были переданы ему или его организации. Для каждой заявки отображаются категория, адрес или координаты, статус, дата создания и фотография проблемы. Исполнитель может принять заявку в работу, изменить её статус, а после выполнения загрузить фотографию результата. Это позволяет подтвердить факт выполнения работ и сохранить историю обработки обращения.

Для исполнителя также предусматривается работа с местоположением заявки. Система хранит координаты обращений и отображает их на карте, что помогает быстрее ориентироваться на местности и планировать выполнение работ.

В системе также предусмотрена возможность выполнения отдельных простых заявок обычными жителями. Если обращение не требует специальных навыков, доступа к оборудованию или участия коммунальных служб, оно может быть доступно для самостоятельного выполнения пользователями. В таком случае житель может выбрать подходящую заявку, выполнить необходимые действия и отправить подтверждение выполнения. Это позволяет быстрее решать несложные проблемы городской среды и повышает вовлечённость граждан в поддержание чистоты и порядка.

Административная часть системы предназначена для контроля и управления заявками. Администратор может просматривать общий список обращений, фильтровать их по статусу, категории, дате, исполнителю или району. Также администратор может назначать исполнителя, изменять статус заявки, отклонять некорректные обращения, просматривать фотографии и историю изменений. Кроме того, административная часть позволяет управлять справочниками категорий, организациями и пользователями.

Серверная часть приложения выполняет роль связующего звена между клиентскими приложениями и базой данных. Она принимает запросы от

пользователей, проверяет права доступа, обрабатывает данные и возвращает результат. Например, при создании заявки сервер получает данные о категории, координатах, описании и фотографии, после чего сохраняет основную информацию в таблицу заявок, а путь к фотографии — в отдельную таблицу фотографий. При изменении статуса заявки сервер фиксирует новое состояние и добавляет запись в историю изменения статусов [9, 10].

Для обмена данными между клиентской и серверной частью используется API. Оно позволяет выполнять регистрацию и авторизацию пользователей, создавать и просматривать заявки, изменять статусы, загружать фотографии, получать список категорий и организаций. Использование API делает систему более гибкой, так как к одной серверной части могут обращаться разные клиенты: мобильное приложение жителя, интерфейс исполнителя и административная панель [10].

В процессе разработки были выделены основные роли пользователей:

- житель;
- исполнитель;
- администратор.

Житель имеет ограниченный набор прав. Он может создавать заявки, просматривать свои обращения и отслеживать изменение их статусов. Исполнитель работает только с назначенными заявками и может изменять их состояние в пределах своих полномочий. Администратор имеет расширенный доступ и управляет всеми основными данными системы.

Для разграничения доступа используется ролевая модель. При входе в систему определяется роль пользователя, после чего ему становятся доступны только разрешённые функции. Такой подход повышает безопасность приложения и исключает возможность выполнения действий, не соответствующих правам пользователя. Например, обычный житель не может назначать исполнителей или изменять статусы чужих заявок, а исполнитель не может управлять справочниками системы.

При проектировании интерфейса основное внимание уделялось удобству работы пользователей с разными ролями и уровнем доступа. Структура экранов и набор доступных действий формировались с учётом типовых сценариев использования системы. Для жителя приоритетом является быстрое создание обращения без лишних действий, поэтому интерфейс должен быть простым и понятным. Для исполнителя важны оперативный просмотр назначенных задач, работа с местоположением заявок и подтверждение результата выполнения. Административная часть, в свою очередь, включает расширенный набор



функций, необходимых для контроля заявок, управления пользователями и анализа работы системы.

Основным объектом приложения является заявка. Она проходит несколько этапов обработки: создание, назначение исполнителя, принятие в работу, выполнение и закрытие. На каждом этапе статус заявки изменяется и сохраняется в системе. Благодаря этому можно проследить весь путь обращения от момента его создания до завершения работ.

Для хранения фотографий используется отдельный механизм. Изображения не хранятся непосредственно в основной таблице заявок. В базе данных сохраняется путь к файлу, а само изображение размещается в файловом хранилище. Такой подход уменьшает нагрузку на базу данных и упрощает работу с медиафайлами. Фотографии могут быть двух типов: до выполнения работ и после выполнения работ [9, 13]

Важной частью приложения является работа с геолокационными данными. При создании заявки пользователь указывает место обнаруженной проблемы городской среды. Для этого могут использоваться два способа: автоматическое определение координат с помощью геолокации мобильного устройства либо ручной выбор точки на карте. Такой подход делает создание заявки более удобным, так как пользователь может быстро зафиксировать своё текущее местоположение или самостоятельно уточнить место проблемы, если оно находится не рядом с ним [2, 3, 7].

Полученные координаты сохраняются вместе с заявкой и используются при её дальнейшей обработке. Наличие геолокационных данных позволяет отображать обращение на карте, упрощает поиск места выполнения работ и помогает исполнителю быстрее ориентироваться на местности. В административной части системы заявки с указанными координатами могут быть представлены в наглядном виде на карте, что облегчает контроль за их распределением по территории города [3, 7].

Кроме того, хранение координат создаёт основу для дальнейшего развития аналитических возможностей системы. На основе накопленных данных можно определять участки города, где обращения возникают чаще всего, анализировать распределение проблем по районам и выявлять зоны, требующие повышенного внимания со стороны коммунальных служб или администрации. Таким образом, геолокация используется не только для указания места конкретной заявки, но и как дополнительный инструмент контроля, планирования и анализа городской среды.

### 2.2.1 Диаграмма Use Case

Диаграмма Use Case используется для описания функциональных возможностей системы с точки зрения пользователей. Она позволяет показать, какие действия могут выполнять отдельные роли и каким образом они взаимодействуют с программным продуктом. Для информационной системы «Чистый город» были выделены три основные роли: житель, исполнитель и администратор [14].

В информационной системе «Чистый город» предусмотрено разграничение пользователей по ролям. Житель фиксирует проблему городской среды и создаёт заявку, исполнитель обрабатывает назначенные обращения и подтверждает выполнение работ, а администратор контролирует процесс обработки заявок, распределяет их между исполнителями и просматривает статистическую информацию. Такое разделение позволяет ограничить доступ к функциям системы в зависимости от задач пользователя и повысить удобство работы с программным продуктом.

Диаграмма вариантов использования информационной системы «Чистый город» представлена в графической части дипломного проекта.

Диаграмма Use Case отражает основные сценарии взаимодействия пользователей с системой. Для жителя предусмотрены функции регистрации, авторизации, создания заявки, просмотра собственных обращений и отслеживания их статуса. При оформлении заявки пользователь выбирает категорию проблемы, добавляет фотографию, указывает местоположение загрязнения и при необходимости оставляет комментарий. После отправки обращения житель может следить за его обработкой.

Исполнитель работает с заявками, назначенными ему или его организации. После входа в систему он может просматривать назначенные заявки, принимать их в работу, изменять статус, загружать фотографию «после» и завершать заявку. Дополнительно исполнитель может использовать координаты заявки для ориентирования на местности.

Администратор имеет расширенные права доступа. Он может просматривать все заявки, выполнять фильтрацию по различным параметрам, назначать исполнителей и изменять статусы заявок. Также администратору доступны функции управления категориями, организациями и пользователями. Кроме того, он может просматривать статистику и отчёты, рассматривать жалобы и блокировать пользователей при необходимости.

## 2.2.2 Диаграмма последовательности

Диаграмма последовательности используется для описания последовательности действий, которые выполняются между пользователями и компонентами системы во время работы приложения. Она позволяет показать, в каком порядке происходит обмен сообщениями между участниками процесса, интерфейсом приложения, серверной частью и базой данных [14].

Для информационной системы «Чистый город» в качестве основного сценария была выбрана последовательность создания и обработки заявки о загрязнении городской среды. Данный сценарий является ключевым, так как именно вокруг заявки строится основная логика работы системы.

Диаграмма последовательности информационной системы «Чистый город» представлена в графической части дипломного проекта.

На данной диаграмме показан порядок взаимодействия между жителем, мобильным приложением, серверной частью, базой данных, файловым хранилищем, администратором и исполнителем.

Последовательность работы системы начинается с авторизации пользователя. Пользователь вводит данные для входа в мобильном приложении, после чего приложение отправляет запрос на серверную часть. Сервер проверяет корректность введенных данных, определяет роль пользователя и при успешной авторизации возвращает необходимые сведения для дальнейшей работы с системой. После этого житель получает доступ к функциям создания и просмотра заявок.

Далее житель формирует новую заявку о проблеме городской среды. Для этого он выбирает категорию обращения, добавляет описание проблемы, прикрепляет фотографию и указывает местоположение. Координаты могут быть определены автоматически с помощью геолокации устройства либо выбраны пользователем вручную на карте. После заполнения формы мобильное приложение передаёт данные заявки на серверную часть через API.

На стороне сервера выполняется проверка полученной информации: проверяется наличие обязательных полей, корректность категории, наличие авторизованного пользователя и допустимость выполняемого действия. После успешной проверки данные заявки сохраняются в базе данных. Фотография, прикрепленная пользователем, передается на сервер и связывается с созданной заявкой. Также в базе данных фиксируются начальный статус обращения, дата создания, сведения о пользователе и запись в истории изменения статусов.

После создания заявка становится доступной для просмотра в административной части системы. Администратор открывает список новых обращений, анализирует информацию по заявке, просматривает описание, фотографию, категорию и местоположение на карте. При необходимости администратор может изменить статус обращения, отклонить некорректную заявку или назначить ответственного исполнителя. Назначение исполнителя сохраняется в базе данных, после чего заявка переходит на следующий этап обработки.

Исполнитель получает доступ к назначенной заявке в мобильном приложении. Он просматривает информацию о проблеме, изучает описание, фотографию и местоположение. После принятия заявки в работу её статус изменяется, что позволяет администрации и жителю отслеживать ход выполнения. Исполнитель выполняет необходимые действия на месте, после чего подтверждает результат в приложении и при необходимости прикрепляет фотографию после устранения проблемы.

На завершающем этапе серверная часть получает данные о выполнении заявки, сохраняет фотографию результата, обновляет статус обращения и добавляет новую запись в историю изменений. После этого заявка считается выполненной, а пользователь и администратор могут увидеть обновлённую информацию о её состоянии. Такая последовательность обеспечивает контроль всех этапов обработки обращения: от создания заявки жителем до подтверждения выполненных работ исполнителем.

### **2.2.3 Диаграмма классов**

Диаграмма классов используется для описания структуры программного продукта и отображает основные классы системы, их атрибуты, методы и связи между ними. В информационной системе «Чистый город» диаграмма классов строится на основе основных сущностей предметной области и структуры базы данных [14].

Диаграмма классов разрабатываемого программного продукта представлена на рисунке 2.4.

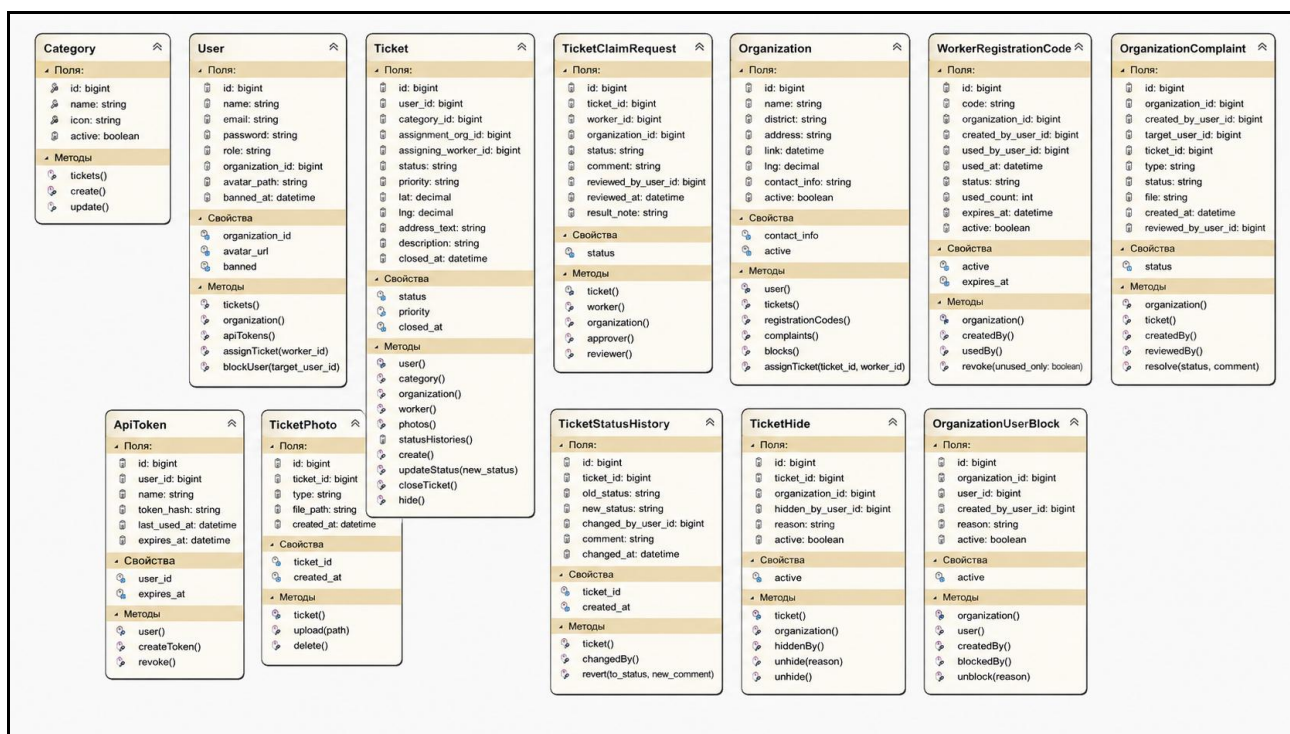


Рисунок 2.4 – Диаграмма классов разрабатываемого программного продукта

Диаграмма классов отображает основные сущности программного продукта, их поля, свойства и методы. Она используется для описания внутренней структуры приложения и показывает, какие классы участвуют в обработке заявок о загрязнении городской среды.

В диаграмме представлены следующие основные классы: Category, User, Ticket, TicketClaimRequest, Organization, WorkerRegistrationCode, OrganizationComplaint, ApiToken, TicketPhoto, TicketStatusHistory, TicketHide и OrganizationUserBlock.

Класс Category предназначен для хранения категорий заявок. Он содержит идентификатор, название категории, иконку и признак активности. С помощью данного класса заявки разделяются по типам проблем.

Класс User описывает пользователя системы. В нём хранятся имя, электронная почта, пароль, роль, идентификатор организации, путь к аватару и сведения о блокировке. Методы класса позволяют получать заявки пользователя, организацию, API-токены, назначать заявки и блокировать пользователей.

Класс Ticket является центральным классом диаграммы. Он описывает заявку о загрязнении городской среды. В классе хранятся данные о пользователе, категории, назначенной организации, исполнителе, статусе, приоритете, координатах, адресе, описании и дате закрытия. Методы класса используются для

получения связанных данных, создания заявки, изменения статуса, закрытия заявки и скрывтия обращения.

Класс Organization хранит сведения об организации-исполнителе. В нём указываются название организации, район, адрес, координаты, контактная информация и признак активности. Методы класса позволяют получать пользователей организации, заявки, регистрационные коды, жалобы, блокировки и назначать исполнителя на заявку.

Класс TicketPhoto предназначен для работы с фотографиями заявок. Он содержит идентификатор заявки, тип изображения, путь к файлу и дату создания. Методы класса позволяют связать фотографию с заявкой, загрузить изображение и удалить его.

Класс TicketStatusHistory используется для хранения истории изменения статусов заявки. В нём фиксируются предыдущий статус, новый статус, пользователь, изменивший статус, комментарий и дата изменения. Этот класс позволяет отслеживать жизненный цикл заявки.

Класс ApiToken предназначен для хранения токенов доступа пользователя. Он содержит идентификатор пользователя, название токена, хэш токена, дату последнего использования и срок действия. Методы класса используются для создания и отзыва токена.

Класс WorkerRegistrationCode хранит регистрационные коды исполнителей. Он используется для привязки работника к организации и содержит код, идентификатор организации, данные о создателе и пользователе, который применил код, дату использования, срок действия и признак активности.

Класс TicketClaimRequest описывает запрос исполнителя на принятие заявки в работу. В нём хранятся идентификаторы заявки, исполнителя и организации, статус запроса, комментарий, пользователь, который рассмотрел запрос, дата рассмотрения и результат.

Класс TicketHide предназначен для скрывтия заявок. Он содержит идентификатор заявки, организации, пользователя, скрывшего заявку, причину скрывтия и признак активности. Данный класс позволяет временно скрывать или возвращать заявку в отображение.

Класс OrganizationComplaint используется для хранения жалоб, связанных с организацией, пользователем или заявкой. В нём содержатся данные о типе жалобы, статусе, описании, связанном пользователе, заявке и пользователе, который рассмотрел жалобу.

Класс OrganizationUserBlock описывает блокировку пользователя в рамках организации. Он хранит идентификатор организации, пользователя, создателя

блокировки, причину и признак активности. С помощью данного класса можно ограничивать взаимодействие пользователя с конкретной организацией.

### **2.3 Тестирование программного продукта**

Тестирование программного продукта — это процесс проверки разработанной системы с целью выявления ошибок, подтверждения корректности работы функций и оценки соответствия программы поставленным требованиям. В рамках данного проекта тестирование проводилось для двух частей системы: мобильного приложения CleanCity и серверной части clean-city-app, включающей REST API и веб-панель администратора [15].

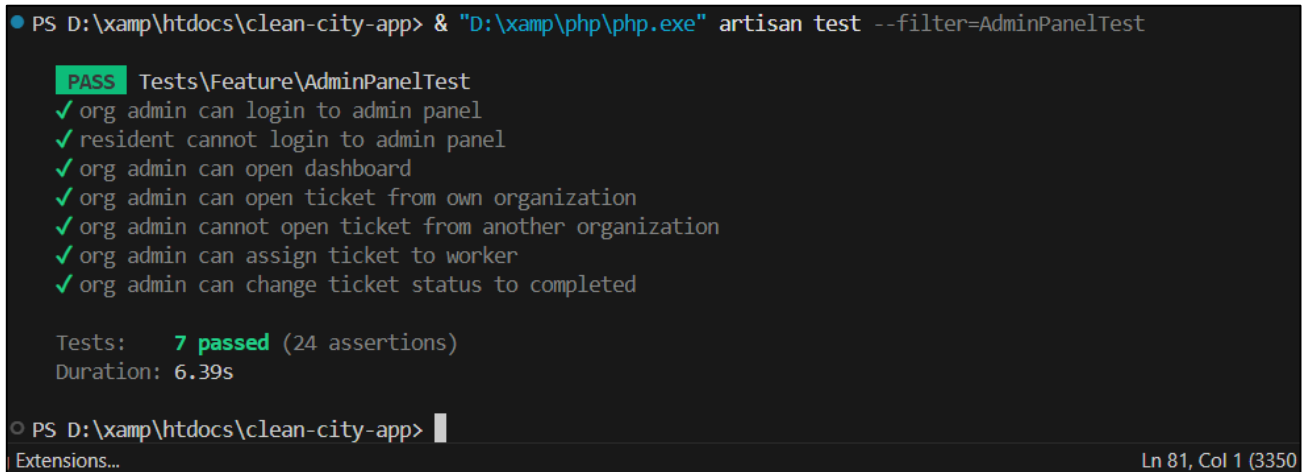
Основной целью тестирования являлась проверка корректной работы ключевых сценариев системы «Чистый город»: регистрация и авторизация пользователей, создание заявок жителями, загрузка фотографий, выбор местоположения, отображение заявок в административной панели, назначение исполнителей, изменение статусов заявок, закрытие заявки с фотографией результата, а также разграничение доступа между ролями пользователя.

Тестирование административной части системы выполнялось с использованием встроенных средств тестирования Laravel и фреймворка PHPUnit. Для проверки были разработаны функциональные тесты, расположенные в каталоге tests/Feature. Запуск тестирования осуществлялся из командной строки PowerShell с помощью команды `php artisan test --filter=AdminPanelTest` [9, 16].

В ходе тестирования проверялись ключевые сценарии работы административной панели: успешная авторизация администратора организации, запрет входа в административную часть для обычного жителя, открытие главной панели администратора, просмотр заявки своей организации, запрет доступа к заявке другой организации, назначение заявки исполнителю, а также изменение статуса заявки на «выполнена». Для изоляции тестов использовалась тестовая база данных, что позволило проверять работу системы без изменения основных данных проекта.

На рисунке представлена консоль с результатами выполнения тестов административной части информационной системы «Чистый город». В результате запуска набора AdminPanelTest все 7 тестов были успешно пройдены. Также было выполнено 24 проверки утверждений, что подтверждает корректную работу основных проверяемых сценариев административной панели.

Общее время выполнения тестов составило 6,39 секунды.



```
PS D:\xampp\htdocs\clean-city-app> & "D:\xampp\php\php.exe" artisan test --filter=AdminPanelTest

PASS Tests\Feature\AdminPanelTest
✓ org admin can login to admin panel
✓ resident cannot login to admin panel
✓ org admin can open dashboard
✓ org admin can open ticket from own organization
✓ org admin cannot open ticket from another organization
✓ org admin can assign ticket to worker
✓ org admin can change ticket status to completed

Tests: 7 passed (24 assertions)
Duration: 6.39s

PS D:\xampp\htdocs\clean-city-app>
```

Рисунок 2.5 Результат выполнения функциональных тестов административной части системы

В процессе проверки использовались следующие виды тестирования:

Функциональное тестирование — проверка основных возможностей системы: вход в приложение, регистрация, создание заявки, просмотр списка заявок, назначение исполнителя, изменение статуса заявки и завершение работы [15].

Интеграционное тестирование — проверка взаимодействия мобильного приложения с серверной частью через API. Особое внимание уделялось передаче токена авторизации, отправке формы заявки, загрузке изображений и получению данных с сервера [15].

Тестирование пользовательского интерфейса — проверка удобства экранов мобильного приложения и веб-панели администратора: корректность отображения кнопок, карточек заявок, сообщений об ошибках и элементов управления.

Тестирование валидации данных — проверка реакции системы на некорректные данные: пустые поля, неправильный email, короткий пароль, отсутствие фотографии или местоположения при создании заявки.

Тестирование безопасности и разграничения прав доступа — проверка того, что пользователь может выполнять только те действия, которые соответствуют его роли. Например, житель может создавать и просматривать свои заявки, исполнитель — работать с назначенными заявками, администратор — управлять заявками и исполнителями [15].



Регрессионное тестирование — повторная проверка основных функций после внесения изменений в интерфейс, карту, кнопку выхода и обработку статусов заявок.

Тестирование проводилось в локальной среде разработки. Серверная часть запускалась на локальном компьютере с использованием Laravel и базы данных проекта. Мобильное приложение проверялось на Android-устройстве или эмуляторе. Для связи мобильного приложения с сервером использовался локальный адрес API либо временный публичный адрес через Cloudflare Tunnel. Веб-панель администратора проверялась через браузер.

В качестве тестовых пользователей использовались роли: житель, исполнитель и администратор. Это позволило проверить работу системы с разных уровней доступа.

Таблица 2.3 — Результаты тестирования программного продукта

| № | Тестируемая функция                            | Ожидаемый результат                                    | Фактический результат                      | Статус  |
|---|------------------------------------------------|--------------------------------------------------------|--------------------------------------------|---------|
| 1 | Проверка работоспособности API                 | Сервер возвращает сообщение о работоспособности        | API возвращает сообщение «API работает»    | Успешно |
| 2 | Регистрация пользователя с корректными данными | Пользователь создаётся, возвращается токен авторизации | Регистрация выполнена                      | Успешно |
| 3 | Регистрация с пустыми или некорректными полями | Вывод сообщения об ошибке                              | Ошибка отображается, данные не сохраняются | Успешно |
| 4 | Вход с корректными данными                     | Пользователь входит в систему                          | Вход выполнен                              | Успешно |
| 5 | Вход с неправильным паролем                    | Система отказывает во входе                            | Отображается сообщение о неверных данных   | Успешно |

| №  | Тестируемая функция                            | Ожидаемый результат                              | Фактический результат                                | Статус  |
|----|------------------------------------------------|--------------------------------------------------|------------------------------------------------------|---------|
| 6  | Отображение главного экрана жителя             | Показываются функции создания и просмотра заявок | Интерфейс соответствует роли жителя                  | Успешно |
| 7  | Отображение главного экрана исполнителя        | Показываются назначенные задачи                  | Интерфейс соответствует роли исполнителя             | Успешно |
| 8  | Загрузка списка категорий                      | Категории загружаются из API                     | Список категорий отображается                        | Успешно |
| 9  | Создание заявки с полными данными              | Заявка сохраняется в базе данных                 | Заявка создана                                       | Успешно |
| 10 | Создание заявки без фотографии                 | Вывод предупреждения пользователю                | Отображается сообщение о необходимости выбрать фото  | Успешно |
| 11 | Создание заявки без указания места             | Вывод предупреждения пользователю                | Отображается сообщение о необходимости выбрать место | Успешно |
| 12 | Выбор точки на карте                           | Координаты передаются в форму заявки             | Местоположение сохранено                             | Успешно |
| 13 | Просмотр раздела «Мои заявки»                  | Пользователь видит только свои заявки            | Список заявок отображается корректно                 | Успешно |
| 14 | Вход администратора в панель управления        | Открывается административная панель              | Доступ предоставлен                                  | Успешно |
| 15 | Просмотр списка заявок в панели администратора | Созданные заявки отображаются в списке           | Заявки доступны для просмотра                        | Успешно |

| №  | Тестируемая функция                 | Ожидаемый<br>результат                           | Фактический<br>результат | Статус  |
|----|-------------------------------------|--------------------------------------------------|--------------------------|---------|
| 16 | Назначение исполнителя на<br>заявку | Заявка получает<br>исполнителя и новый<br>статус | Исполнитель<br>назначен  | Успешно |

## 3 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

### 3.1 Установка мобильного приложения

Для установки мобильного приложения CleanCity необходимо перейти на официальный сайт системы «Чистый город» с помощью любого браузера на устройстве Android. Главная страница сайта представлена на рисунке 3.1.

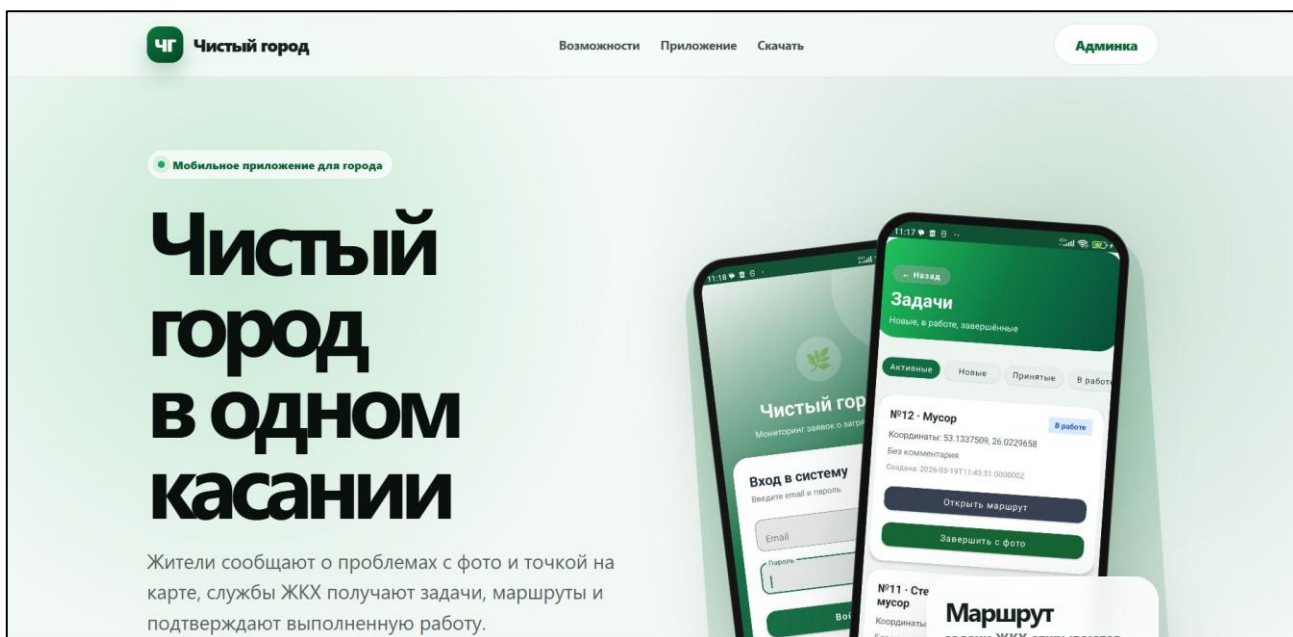


Рисунок 3.1 – Главная страница сайта «Чистый город»

На главной странице сайта расположен краткий обзор возможностей системы и кнопка «Скачать», которая ведёт к разделу загрузки. Для перехода к разделу загрузки необходимо прокрутить страницу вниз или нажать кнопку «Скачать» в верхней части страницы.

Раздел загрузки приложения представлен на рисунке 3.2.

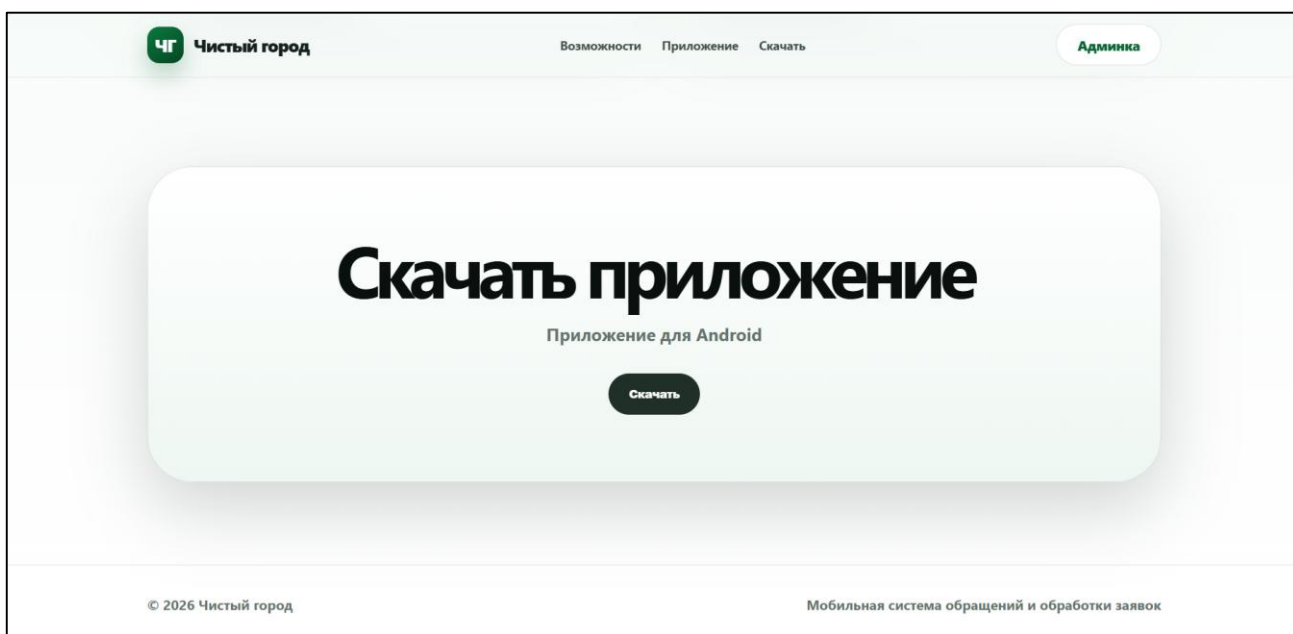


Рисунок 3.2 – Раздел загрузки приложения на сайте

В разделе загрузки необходимо нажать кнопку «Скачать». Браузер начнёт загрузку установочного файла формата APK. После завершения загрузки файл будет доступен в папке «Загрузки» устройства.

Перед установкой загруженного APK-файла необходимо разрешить установку приложений из неизвестных источников. Для этого следует открыть «Настройки» устройства, перейти в раздел «Безопасность» или «Конфиденциальность», найти пункт «Установка неизвестных приложений» или «Разрешить установку из этого источника» и активировать его для используемого браузера. Порядок шагов может отличаться в зависимости от версии Android и производителя устройства.

После предоставления разрешения необходимо открыть загруженный APK-файл через файловый менеджер или уведомление о завершении загрузки в браузере. Откроется системное окно установки приложения, представленное на рисунке

3.3. Необходимо нажать кнопку «Установить» и дождаться завершения процесса.

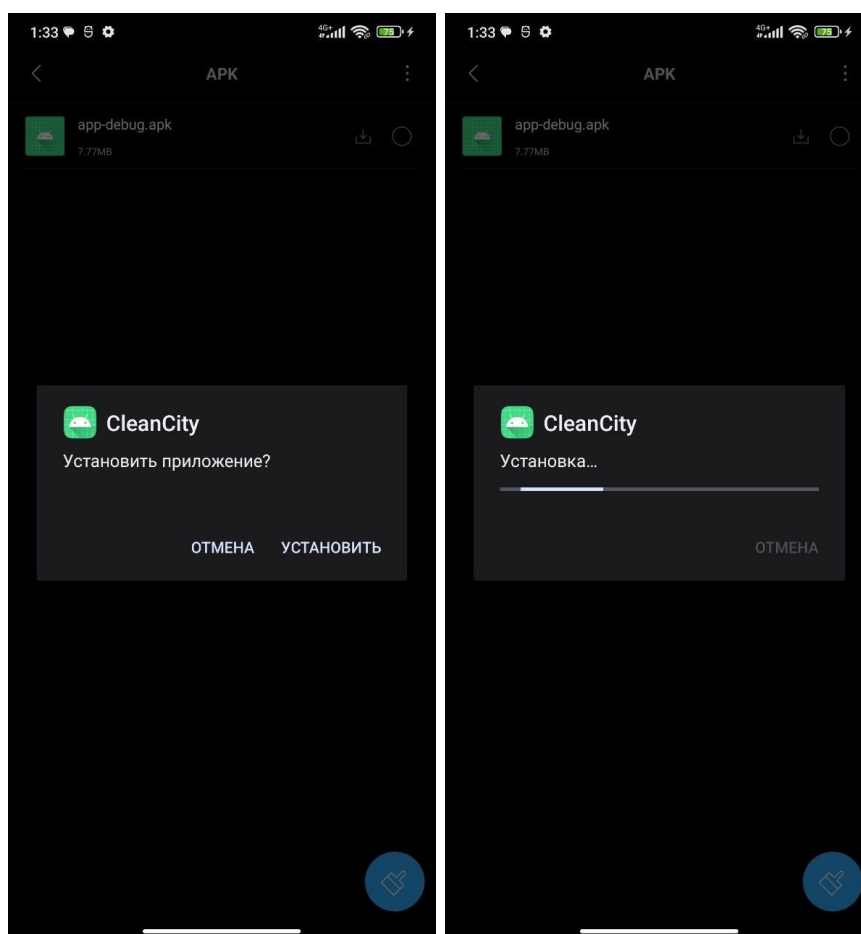


Рисунок 3.3 – Системное окно установки APK-файла

После успешной установки значок приложения CleanCity появится на главном экране или в списке приложений устройства. Для запуска необходимо нажать на значок. При первом использовании функций создания заявки приложение может запросить разрешение на доступ к местоположению и изображениям устройства — эти разрешения необходимы для указания точки проблемы и прикрепления фотографии.

### 3.2 Вход в систему

После запуска приложения на экране отображается экран авторизации, представленный на рисунке 3.4. Пользователь должен ввести адрес электронной почты в поле «Email» и пароль в поле «Пароль», после чего нажать кнопку «Войти». Если введенные данные корректны, система определяет роль

пользователя и открывает соответствующий главный экран. При вводе неверных учётных данных под полями появляется сообщение об ошибке.

Рисунок 3.4 – Экран авторизации мобильного приложения

Если учётная запись ещё не создана, необходимо нажать ссылку «Зарегистрироваться». Откроется форма регистрации, представленная на рисунке 3.5. В форме следует заполнить поля: «Имя», «Email», «Пароль» и «Подтверждение пароля». После нажатия кнопки «Зарегистрироваться» система создаёт учётную запись с ролью «Житель» и выполняет автоматический вход. При несовпадении паролей или некорректном формате email под соответствующим полем выводится поясняющее сообщение.

Рисунок 3.5 – Форма регистрации нового пользователя

Для входа в веб-панель администратора необходимо открыть браузер и перейти по адресу сервера. Страница авторизации веб-панели представлена на рисунке 3.6. Администратор вводит учётные данные и нажимает кнопку «Войти». После успешного входа открывается главная страница панели управления.

Рисунок 3.6 – Страница авторизации веб-панели администратора

Страница авторизации отделяет доступ к административной части от мобильного приложения и позволяет открыть панель управления только пользователям с соответствующей ролью.

### 3.3 Диалоговое окно жителя

После авторизации с ролью «Житель» открывается главный экран приложения, представленный на рисунке 3.7. На экране расположены основные действия: создание новой заявки и переход к списку собственных обращений.

Рисунок 3.7 – Главный экран приложения в режиме жителя

Для подачи заявки необходимо нажать кнопку «Создать заявку» на главном экране. Откроется форма создания заявки, представленная на рисунке 3.8.

Рисунок 3.8 – Форма создания заявки

Форма содержит следующие поля:

- «Категория» — обязательное поле. Нажать на поле и выбрать тип проблемы из выпадающего списка: «Мусор», «Переполненная урна», «Несанкционированная свалка», «Граффити» и другие;
- «Фотография» — обязательное поле. Нажать кнопку «Добавить фото» и сделать снимок камерой или выбрать изображение из галереи устройства;
- «Местоположение» — обязательное поле. Координаты определяются автоматически через GPS. При необходимости уточнить точку нажать кнопку «Указать на карте» (рисунок 3.9);
- «Комментарий» — необязательное поле. Текстовое описание проблемы до 500 символов.

Рисунок 3.9 – Экран выбора местоположения на карте

После заполнения всех обязательных полей необходимо нажать кнопку «Отправить» внизу формы. При успешной отправке появляется уведомление



«Заявка успешно отправлена» и происходит переход в раздел «Мои заявки». Если обязательное поле не заполнено, оно выделяется красной рамкой с поясняющим сообщением.

Раздел «Мои заявки» открывается через карточку на главном экране и отображает список всех обращений текущего пользователя. Список заявок представлен на рисунке 3.10.

Рисунок 3.10 – Раздел «Мои заявки»

Каждая карточка содержит: категорию, текущий статус, описание, сведения о наличии места и дату создания. Статус обозначается цветовой меткой: серый – «Новая», синий – «В работе», зелёный – «Выполнено», красный – «Отклонено».

### **3.4 Диалоговое окно исполнителя**

После авторизации с ролью «Исполнитель» открывается главный экран исполнителя, представленный на рисунке 3.11. На экране отображается кнопка перехода к назначенным задачам.

Рисунок 3.11 – Главный экран приложения в режиме исполнителя

Список обращений отображает заявки, назначенные данному исполнителю или его организации. Каждая карточка показывает номер заявки, категорию, статус, описание и сведения о месте. В карточке доступны действия исполнителя, представленные на рисунке 3.12.

Рисунок 3.12 – Карточка назначенной заявки для исполнителя

В карточке заявки исполнителю доступны следующие действия:

- «Принять» – кнопка активна для назначенных заявок. После нажатия статус изменяется на «Принята», а житель видит обновлённое состояние обращения в списке своих заявок;
- «Завершить» – активна для заявок «В работе». При нажатии открывается экран загрузки фотографии результата. Прикрепление фото обязательно. После загрузки статус изменяется на «Выполнено»;
- «Отклонить» – позволяет отклонить заявку с обязательным указанием причины в поле комментария.

Координаты заявки используются для отображения местоположения обращения на карте. Наличие координат помогает быстрее найти место выполнения работ и перейти к нужному обращению. Пример отображения активных точек на карте представлен на рисунке 3.13.

Рисунок 3.13 – Пример отображения активных точек на карте

После авторизации в веб-панели открывается главная страница, представленная на рисунке 3.14. Интерфейс состоит из боковой панели навигации слева и основной рабочей области. В верхней части страницы отображаются заголовок текущего раздела и кнопка выхода из системы.

Боковая панель навигации содержит следующие разделы: «Дашборд», «Заявки», «Карта заявок», «Организации», «Сотрудники», «Пользователи», «Запросы», «Новости», «Купоны», «Баллы», «Аналитика».

Рисунок 3.14 – Главная страница (дашборд) веб-панели администратора

Рассмотрим подробно каждый раздел, доступный администратору в системе.

Раздел «Дашборд» открывается после входа и содержит сводную статистику системы. В верхней части отображаются блоки-счётчики: общее число пользователей, количество заблокированных аккаунтов, число организаций и администраторов ЖКХ. Ниже расположен справочник

организаций с количеством сотрудников и заявок по каждой из них, а также список последних зарегистрированных пользователей.

Раздел «Заявки» является основным рабочим инструментом. Страница представлена на рисунке 3.15. Таблица содержит колонки: номер заявки, категория, адрес, статус, приоритет, дата создания, организация и исполнитель. Над таблицей расположена панель фильтров: по статусу, категории, организации и диапазону дат.

Рисунок 3.15 – Раздел «Заявки» веб-панели администратора

Для просмотра подробной информации о заявке необходимо нажать на строку таблицы. Откроется страница деталей заявки (рисунок 3.16), где отображаются фотография «до», фотография «после» (при наличии), точка на карте и полная история статусов.

Рисунок 3.16 – Страница детальной информации заявки

Для назначения исполнителя необходимо нажать кнопку «Назначить» и в открывшемся окне выбрать организацию и конкретного сотрудника. После подтверждения статус заявки изменяется на «Назначена», а исполнитель видит обращение в списке своих заявок. Для изменения статуса вручную необходимо нажать кнопку «Изменить статус», выбрать новое значение и добавить комментарий. Кроме этого, доступны кнопки «Скрыть заявку» (скрывает её от жителя) и «Восстановить».

Раздел «Карта заявок» отображает все заявки на интерактивной карте в виде маркеров. Страница представлена на рисунке 3.17. Цвет маркера соответствует текущему статусу заявки. При нажатии на маркер отображается всплывающая карточка с краткой информацией. Карта поддерживает масштабирование и перемещение, а при большом количестве близко расположенных заявок маркеры объединяются в кластеры с числом обращений.

Рисунок 3.17 – Раздел «Карта заявок»

Раздел «Организации» предназначен для просмотра справочника организаций-исполнителей. Страница представлена на рисунке 3.18. Таблица содержит: название, адрес, число администраторов, число сотрудников и число назначенных заявок. Справочник организаций является фиксированным и не создаётся вручную – он уже внесён в систему. При нажатии на строку организации открывается детальная страница с перечнем её сотрудников и статистикой заявок.

Рисунок 3.18 – Раздел «Организации» веб-панели администратора

Раздел «Сотрудники» позволяет просматривать список работников всех организаций, а также управлять регистрационными кодами для исполнителей. Для создания нового кода необходимо нажать кнопку «Создать код», указать организацию и срок действия. Сгенерированный код передаётся сотруднику для регистрации в мобильном приложении в роли исполнителя. Для деактивации кода необходимо нажать соответствующую кнопку в строке кода.

Раздел «Пользователи» отображает список всех зарегистрированных пользователей системы. Страница представлена на рисунке 3.19. Таблица содержит: имя, email, роль, дату регистрации и статус учётной записи. Для блокировки пользователя необходимо нажать кнопку «Заблокировать» и ввести причину. Заблокированный пользователь не сможет выполнять действия в системе. Для снятия блокировки нажать кнопку «Разблокировать» в карточке пользователя.

Рисунок 3.19 – Раздел «Пользователи» веб-панели администратора

Раздел «Новости» позволяет публиковать информационные материалы, которые отображаются в мобильном приложении. Для создания новости необходимо нажать кнопку «Создать новость», заполнить заголовок, текст и при необходимости прикрепить изображение, после чего нажать «Опубликовать». Для редактирования или удаления существующей новости использовать соответствующие кнопки в строке таблицы.

Раздел «Купоны» предназначен для управления системой поощрений пользователей. Администратор может создавать вознаграждения, указывая название, описание и стоимость в баллах. Пользователи получают баллы за активность в системе и могут обменивать их на доступные купоны или призы.

Раздел «Баллы» позволяет просматривать историю начисления и списания баллов для каждого пользователя, а также вручную корректировать баланс. Для корректировки необходимо выбрать пользователя, ввести количество баллов (положительное для начисления, отрицательное для списания) и указать причину. Для просмотра полной истории баллов конкретного пользователя нажать кнопку «История» в его строке.

Раздел «Запросы» содержит запросы исполнителей на самостоятельное принятие заявок в работу без назначения администратором. Страница отображает: заявку, исполнителя, организацию и дату запроса. Для рассмотрения запроса необходимо нажать кнопку «Одобрить» или «Отклонить» с указанием комментария.

Раздел «Аналитика» предоставляет инструменты для анализа работы системы. Страница представлена на рисунке 3.20. В верхней части расположены поля выбора периода. После выбора дат и нажатия «Применить» обновляются следующие блоки: динамика поступления заявок (линейный график), распределение по категориям (круговая диаграмма) и показатели эффективности организаций. Для сохранения данных необходимо нажать кнопку «Экспорт» и выбрать формат – CSV или PDF.

Рисунок 3.20 – Раздел «Аналитика» веб-панели администратора

Для выхода из веб-панели необходимо нажать кнопку выхода в верхней части интерфейса. Для выхода из мобильного приложения необходимо нажать кнопку «Выйти» на главном экране.

## 4 ЭКОНОМИЧЕСКИЙ РАЗДЕЛ

### 4.1 Расчет прогнозируемой цены программного средства и прибыли от внедрения программного продукта

В данном разделе рассчитываются прогнозируемая цена программного средства и прибыль от реализации информационной системы «Чистый город». Программный продукт предназначен для приема заявок жителей о загрязнении городской среды, передачи геолокации и фотографий, назначения исполнителей, контроля статусов, ведения истории обработки обращений, начисления баллов, публикации новостей и формирования аналитики в веб-панели администратора [17].

Расчет выполнен для периода разработки с 01.05.2026 по 15.05.2026. В указанном интервале принято 10 рабочих дней и 79 часов работы персонального компьютера. Нормативные и стоимостные показатели для экономического расчета приняты по данным 2025 года.

Исходные показатели для расчета приведены в таблице 4.1.

Таблица 4.1 - Исходные данные для экономического расчета

| Показатель                                     | Обозн. | Значение    |
|------------------------------------------------|--------|-------------|
| Базовая ставка для оплаты труда                | БС     | 270,00 руб. |
| Тарифный коэффициент исполнителя               | ТК     | 3,5         |
| Коэффициент повышения тарифного оклада         | Кп     | 1,2         |
| Количество рабочих дней в мае 2026 года        | Т      | 20 дн.      |
| Трудоемкость разработки                        | Т0     | 10 чел.-дн. |
| Время работы компьютера за период разработки   | t      | 79 ч.       |
| Расчетная норма рабочего времени за 2025 год   | ФРВ    | 2007 ч.     |
| Ставка взносов в ФСЗН                          | %ФСЗН  | 34%         |
| Ставка страховых взносов от несчастных случаев | %стр   | 0,6%        |

| Показатель | Обозн. | Значение |
|------------|--------|----------|
| Ставка НДС | Стндс  | 20%      |

По статье «Затраты на материалы и комплектующие» (Змат) отражаются расходы на бумагу, носитель для резервного копирования, средства специального назначения и канцелярские материалы, необходимые при разработке и оформлении программного средства. Затраты рассчитываются по формуле: [17]

$$\text{Змат} = \Sigma(\text{Нрасх} * \text{Цед}), \text{руб.} \quad (4.1)$$

$$\text{Змат} = (1 * 15,00) + (1 * 25,00) + (1 * 80,00) + (1 * 20,00) = 140,00 \text{ руб.}$$

По статье «Топливо и энергия на технологические цели» (ЗТЭР) отражаются затраты на электроэнергию, необходимую для работы персонального компьютера и оргтехники. Затраты по данной статье рассчитываются по формуле:

$$\text{ЗТЭР} = \text{Рэ} * \text{п} * \text{t}, \text{руб.} \quad (4.2)$$

$$\text{ЗТЭР} = 0,40 * 0,30 * 79 = 9,48 \text{ руб.}$$

где Рэ - расход электроэнергии для работы компьютера, кВт; п - тариф на электроэнергию, руб.; t - время работы компьютера за период разработки, ч.

Статья «Заработная плата производственных рабочих» включает месячный тарифный оклад исполнителя, тарифный оклад за период разработки, а также стимулирующие и компенсирующие выплаты.

$$\Sigma \text{ТО}_{\text{мес}} = \text{БС} * \text{ТК} * \text{Кп}, \text{руб.} \quad (4.3)$$

$$\Sigma \text{ТО}_{\text{мес}} = 270,00 * 3,5 * 1,2 = 1134,00 \text{ руб.}$$

$$\Sigma \text{ТО} = \Sigma \text{ТО}_{\text{мес}} / \text{T} * \text{T0}, \text{руб.} \quad (4.4)$$

$$\Sigma \text{ТО} = 1134,00 / 20 * 10 = 567,00 \text{ руб.}$$

$$\Sigma \text{Вск} = (\Sigma \text{ТО}_{\text{мес}} * 25\%) / \text{T} * \text{T0}, \text{руб.} \quad (4.5)$$

$$\Sigma \text{Вск} = (1134,00 * 25\%) / 20 * 10 = 141,75 \text{ руб.}$$

$$\Phi \text{ЗП} = \Sigma \text{ТО} + \Sigma \text{Вск}, \text{руб.} \quad (4.6)$$

$$\Phi \text{ЗП} = 567,00 + 141,75 = 708,75 \text{ руб.}$$



Обязательные страховые взносы в бюджет государственного внебюджетного фонда социальной защиты населения и страховые взносы от несчастных случаев рассчитываются в процентном отношении к фонду заработной платы: [18]

$$\text{Отчфсзн} = \text{ФЗП} * 34\% / 100, \text{ руб.} \quad (4.7)$$

$$\text{Отчфсзн} = 708,75 * 34\% / 100 = 240,98 \text{ руб.}$$

$$\text{Отчстрах} = \text{ФЗП} * 0,6\% / 100, \text{ руб.} \quad (4.8)$$

$$\text{Отчстрах} = 708,75 * 0,6\% / 100 = 4,25 \text{ руб.}$$

Общепроизводственные и общехозяйственные расходы, связанные с разработкой программного продукта, рассчитываются по нормативу от тарифного оклада исполнителя за период разработки: [17]

$$\text{ОПР} = \Sigma \text{ТО} * 10\% / 100, \text{ руб.} \quad (4.9)$$

$$\text{ОПР} = 567,00 * 10\% / 100 = 56,70 \text{ руб.}$$

$$\text{ОХР} = \Sigma \text{ТО} * 20\% / 100, \text{ руб.} \quad (4.10)$$

$$\text{ОХР} = 567,00 * 20\% / 100 = 113,40 \text{ руб.}$$

Материальные затраты и общая сумма затрат на разработку программного продукта определяются следующим образом:

$$\text{МЗ} = \text{Змат} + \text{ЗТЭР}, \text{ руб.}$$

$$\text{МЗ} = 140,00 + 9,48 = 149,48 \text{ руб.}$$

$$\text{Спр} = \text{МЗ} + \text{ФЗП} + \text{Отчфсзн} + \text{Отчстрах} + \text{ОПР} + \text{ОХР}, \text{ руб.} \quad (4.11)$$

$$\text{Спр} = 149,48 + 708,75 + 240,98 + 4,25 + 56,70 + 113,40 = 1273,56 \text{ руб.}$$

Кроме того, организация-разработчик несет затраты на сопровождение и адаптацию программного продукта, а также коммерческие расходы, связанные с подготовкой продукта к внедрению.

$$\text{Задап} = \text{Спр} * 6\% / 100, \text{ руб.} \quad (4.12)$$

$$\text{Задап} = 1273,56 * 6\% / 100 = 76,41 \text{ руб.}$$

$$\text{КР} = \text{Спр} * 3\% / 100, \text{ руб.}$$

$$КР = 1273,56 * 3\% / 100 = 38,21 \text{ руб.}$$

$$Сп = Спр + Задап + КР, \text{ руб.} \quad (4.13)$$

$$Сп = 1273,56 + 76,41 + 38,21 = 1388,18 \text{ руб.}$$

Свободная отпускная цена определяется на основании цены разработчика с учетом рентабельности и НДС [18].

$$П = Сп * Р / 100, \text{ руб.} \quad (4.14)$$

$$П = 1388,18 * 25\% / 100 = 347,05 \text{ руб.}$$

$$\text{НДС} = (Сп + П) * 20\% / 100, \text{ руб.} \quad (4.15)$$

$$\text{НДС} = (1388,18 + 347,05) * 20\% / 100 = 347,05 \text{ руб.}$$

$$Цотп = Сп + П + \text{НДС}, \text{ руб.} \quad (4.16)$$

$$Цотп = 1388,18 + 347,05 + 347,05 = 2082,28 \text{ руб.}$$

По результатам проведенных расчетов общая сумма затрат на разработку программного продукта составила 1273,56 руб., полная себестоимость - 1388,18 руб. При уровне рентабельности 25% прибыль от реализации программного средства составила 347,05 руб. С учетом НДС прогнозируемая отпускная цена программного продукта составляет 2082,28 руб.

## **4.2 Обоснование экономической целесообразности проведения мероприятия**

Внедрение информационной системы «Чистый город» является экономически целесообразным, так как программное средство сокращает время приема, распределения и контроля заявок о загрязнении городской среды. До внедрения системы обращения жителей могут поступать по телефону, через мессенджеры, социальные сети или в устной форме, что усложняет фиксацию точного места проблемы, сохранение фотографий, передачу заявки исполнителю и контроль сроков выполнения работ [17].

Разработанная система объединяет данные в единой базе: житель создает заявку с фотографией и геолокацией, администратор назначает исполнителя, исполнитель меняет статус и подтверждает выполнение работы фотографией после уборки. Веб-панель администратора позволяет просматривать заявки,

организации, пользователей, запросы исполнителей, карту обращений, новости, баллы и аналитические отчеты.

Экономический эффект выражается в сокращении трудозатрат сотрудника, который принимает и контролирует обращения. Операции поиска заявки, проверки статуса, назначения исполнителя, подготовки сводной информации и контроля результата выполняются быстрее, чем при использовании разрозненных каналов связи и ручных таблиц. Освободившееся рабочее время может быть направлено на контроль качества уборки, анализ проблемных зон и работу с повторными обращениями.

По результатам расчетов полная себестоимость программного продукта составила 1388,18 руб., прибыль при уровне рентабельности 25% - 347,05 руб., прогнозируемая отпускная цена с учетом НДС - 2082,28 руб. Указанные затраты являются обоснованными, так как система повышает прозрачность обработки обращений, снижает вероятность потери информации и ускоряет взаимодействие жителей, исполнителей и администрации.

#### **4.3 Расчет капитальных вложений в создание и разработку программного продукта**

Общие капитальные вложения (КВ) потребителя, связанные с приобретением, внедрением и использованием программного продукта, рассчитываются по формуле:

$$КВ = Кпр + Кпк + Кос + Кис, \text{ руб.} \quad (4.17)$$

$$КВ = 2082,28 + 0,00 + 50,00 + 416,46 = 2548,74 \text{ руб.}$$

где  $Кпр$  - затраты пользователя на приобретение программного средства по отпускной цене с НДС;  $Кпк$  - затраты пользователя на приобретение оргтехники;  $Кос$  - затраты пользователя на освоение компьютера;  $Кис$  - затраты на внедрение, обучение, дополнительные настройки и первичное сопровождение программного продукта. В расчете принято, что используется имеющееся оборудование, поэтому  $Кпк = 0,00$  руб.;  $Кис = Цотп \times 20\% = 416,46$  руб.

По результатам расчета общие капитальные вложения потребителя составили 2548,74 руб. Основную часть затрат составляет приобретение программного средства по отпускной цене с НДС - 2082,28 руб. Дополнительные расходы включают освоение компьютера и внедрение программного продукта.

#### 4.4 Расчет экономической эффективности внедрения программного продукта у потребителя

Экономическая эффективность внедрения информационной системы «Чистый город» определяется за счет сокращения затрат рабочего времени сотрудника администрации или диспетчера, выполняющего прием, регистрацию, распределение и контроль заявок о загрязнении городской среды.

До внедрения программного продукта часть операций выполнялась вручную: заявка фиксировалась в сообщениях или таблицах, адрес уточнялся отдельно, фотография хранилась вне единой базы, а статус исполнения приходилось проверять через исполнителей. После внедрения системы данные заявки, фото, геолокация, исполнитель, статус и история обработки хранятся в едином программном продукте.

Рассчитаем сокращение затрат рабочего времени. Продолжительность рабочего дня составляет 8 часов. Экономия рабочего времени принимается равной 12 минутам за каждый час работы, то есть 0,2 часа на один час работы.

$$\text{Купл} = \text{Эрвч} / 60 \quad (4.18)$$

$$\text{Купл} = 12 / 60 = 0,20$$

$$\text{Эврг} = \text{Купл} * \text{ФРВ}, \text{ ч.} \quad (4.19)$$

$$\text{Эврг} = 0,20 * 2007 = 401,4 \text{ ч.}$$

где Эрвч - экономия рабочего времени за час, мин.; ФРВ - фонд рабочего времени одного работника за год по расчетной норме 2025 года.

$$\text{Эч} = \text{Эврг} / \text{ФРВ}, \text{ чел.} \quad (4.20)$$

$$\text{Эч} = 401,4 / 2007 = 0,20 \text{ чел.}$$

$$\text{ТОч} = \Sigma \text{ТОмес} / (\text{ФРВ} / 12), \text{ руб.} \quad (4.21)$$

$$\text{ТОч} = 1134,00 / (2007 / 12) = 6,78 \text{ руб.}$$

$$\text{Эзп} = \text{ТОч} * \text{Эврг} * \text{Кск} * \text{Ксоц}, \text{ руб.} \quad (4.22)$$

$$\text{Эзп} = 6,78 * 401,4 * 1,3 * 1,346 = 4762,07 \text{ руб.}$$

В результате внедрения информационной системы годовая экономия рабочего времени составляет 401,4 ч., что соответствует условному сокращению

трудозатрат на 0,20 чел. Относительная экономия затрат на заработную плату составляет 4762,07 руб.

#### **4.4.1 Расчет срока окупаемости**

Срок окупаемости показывает, за какой период капитальные вложения, связанные с приобретением и внедрением программного продукта, будут возмещены за счет полученного экономического эффекта.

$$\text{Ток} = \text{КВ} / \text{Эзп, г.} \quad (4.23)$$

$$\text{Ток} = 2548,74 / 4762,07 = 0,54 \text{ г.}$$

Срок окупаемости капитальных вложений составляет 0,54 года, или примерно 6,4 месяца. Это означает, что затраты на приобретение и внедрение программного продукта будут возмещены менее чем за один год за счет экономии затрат на заработную плату.

#### **4.4.2 Расчет возможного роста производительности труда**

Возможный рост производительности труда определяется за счет сокращения времени, необходимого для обработки заявок после внедрения информационной системы. Для расчета принимается, что до внедрения системы выполнение условного объема операций занимало 1 час, а после внедрения - 0,8 часа.

$$\Delta \text{ПТ}_{\text{вр}} = T_0 / T_6 * 100, \% \quad (4.24)$$

$$\Delta \text{ПТ}_{\text{вр}} = 0,8 / 1 * 100 = 80\%$$

Полученное значение показывает, что после внедрения системы на выполнение того же объема работы требуется 80% прежнего времени. Экономия рабочего времени составляет 20%.

$$\Delta \text{ПТ} = \text{Эч} / (\text{Ч} - \text{Эч}) * 100, \% \quad (4.25)$$

$$\Delta \text{ПТ} = 0,20 / (1 - 0,20) * 100 = 25\%$$

Следовательно, возможный рост производительности труда сотрудника, обрабатывающего заявки, после внедрения информационной системы «Чистый город» составляет 25%. Рост достигается за счет автоматизации приема

обращений, хранения фотографий и геолокации, назначения исполнителей и контроля статусов.

#### **4.5 Техничко-экономические показатели внедрения мероприятия. Выводы по расчетам**

Для обобщения результатов расчетов основные технико-экономические показатели внедрения информационной системы «Чистый город» сведены в таблицу 4.2.

Таблица 4.2 - Техничко-экономические показатели внедрения мероприятия

| Наименование показателя                        | Об.  | Ед. изм. | Результат |
|------------------------------------------------|------|----------|-----------|
| 1. Полная себестоимость                        | Сп   | руб.     | 1388,18   |
| 2. Прибыль от реализации программного продукта | П    | руб.     | 347,05    |
| 3. Свободная отпускная цена                    | Цотп | руб.     | 2082,28   |
| 4. Сумма капитальных вложений                  | КВ   | руб.     | 2548,74   |
| 5. Экономия рабочего времени за год            | Эврг | ч.       | 401,4     |
| 6. Экономия затрат на заработную плату за год  | Эзп  | руб.     | 4762,07   |
| 7. Срок окупаемости программного средства      | Ток  | лет      | 0,54      |
| 8. Рост производительности труда               | ПТ   | %        | 25        |

По результатам расчетов установлено, что внедрение информационной системы «Чистый город» является экономически целесообразным. Полная себестоимость программного продукта составляет 1388,18 руб., прибыль от реализации - 347,05 руб., а свободная отпускная цена с учетом прибыли и налога на добавленную стоимость - 2082,28 руб.

Общая сумма капитальных вложений, необходимых для приобретения, освоения и внедрения программного продукта, составляет 2548,74 руб. За счет автоматизации обработки обращений достигается экономия рабочего времени

401,4 ч. в год. Относительная экономия численности составляет 0,20 чел., при этом фактическое сокращение персонала не предусматривается.

Годовая экономия затрат на заработную плату составляет 4762,07 руб. Срок окупаемости капитальных вложений равен 0,54 года, или примерно 6,4 месяца. Возможный рост производительности труда составляет 25%. Полученные показатели подтверждают эффективность внедрения разработанного программного продукта.

## **5 РАЗДЕЛ ПО ОХРАНЕ ТРУДА**

### **5.1 Требования по охране труда по окончании работы и в аварийных ситуациях с ПК и множительной техникой**

Работа с персональным компьютером, принтером, сканером, копировальным аппаратом и другой множительной техникой требует соблюдения правил охраны труда не только в процессе выполнения производственных операций, но и по окончании работы. Завершающий этап является важным, поскольку неправильное отключение оборудования, оставление включенных приборов без присмотра, нарушение порядка уборки рабочего места или игнорирование признаков неисправности могут привести к поражению электрическим током, перегреву техники, возгоранию, повреждению оборудования, а также к травмированию работника [19].

По окончании работы пользователь обязан завершить все выполняемые операции на персональном компьютере: сохранить рабочие файлы, закрыть используемые программы, завершить сеанс работы в информационной системе, при необходимости выйти из учетной записи. Завершение работы ПК должно выполняться штатным способом через операционную систему. Не допускается отключать компьютер путем выдергивания вилки из розетки или выключения сетевого фильтра без предварительного завершения работы, так как это может привести к потере данных, повреждению программного обеспечения и возникновению неисправностей.

После завершения работы компьютер, монитор, принтер, сканер, копировальный аппарат и иные периферийные устройства необходимо отключить от сети, если это предусмотрено инструкцией по эксплуатации и локальными правилами организации. Важно убедиться, что оборудование не находится в режиме печати, копирования, сканирования или обновления программного обеспечения. Если техника подключена через сетевой фильтр или источник бесперебойного питания, отключение должно выполняться в установленной последовательности.

Рабочее место необходимо привести в порядок. Документы, носители информации, канцелярские принадлежности и расходные материалы следует убрать в специально отведенные места. Нельзя оставлять на столе лишние бумаги, особенно рядом с нагревающимися элементами оборудования, розетками, блоками питания, вентиляционными отверстиями системного блока, принтера или копировального аппарата. Загромождение рабочего места



повышает риск случайного повреждения техники, падения предметов, короткого замыкания и возгорания.

Особое внимание следует уделить множительной технике. После окончания работы с принтером или копировальным аппаратом необходимо проверить, не осталось ли внутри аппарата застрявшей бумаги, открытых крышек, неиспользованных документов или посторонних предметов. При наличии сообщения об ошибке на дисплее устройства работник должен действовать в соответствии с инструкцией по эксплуатации. Не допускается самостоятельно разбирать аппарат, вскрывать блоки с тонером, извлекать детали, находящиеся под напряжением, или устранять неисправности, если такие действия не входят в обязанности работника.

При работе с лазерными принтерами и копировальными аппаратами необходимо учитывать, что отдельные элементы оборудования могут нагреваться до высокой температуры. Поэтому после завершения печати или копирования нельзя прикасаться к внутренним узлам аппарата сразу после работы. Удаление застрявшей бумаги допускается только после отключения устройства и с соблюдением указаний производителя. Если бумага застряла в труднодоступной части аппарата, следует обратиться к ответственному лицу или специалисту по обслуживанию.

По окончании работы запрещается оставлять включенными электроприборы, если это не предусмотрено технологическим процессом. Также запрещается оставлять без присмотра оборудование с признаками неисправности: запахом гари, искрением, необычным шумом, перегревом корпуса, повреждением кабеля, нестабильной работой дисплея или повторяющимися ошибками. Обо всех выявленных неисправностях необходимо сообщить непосредственному руководителю, специалисту по охране труда или ответственному за эксплуатацию оборудования.

Важным требованием является соблюдение электробезопасности. Работник не должен касаться вилок, розеток, сетевых кабелей и выключателей мокрыми руками. Не допускается тянуть кабель за провод при отключении устройства от сети — необходимо держаться за корпус вилки. При обнаружении повреждения изоляции проводов, трещин на вилке, искрения розетки или нестабильного питания эксплуатация оборудования должна быть немедленно прекращена [19].

В аварийных ситуациях работник обязан действовать быстро, но без паники. К аварийным ситуациям при работе с ПК и множительной техникой относятся: возгорание оборудования или электропроводки, запах дыма или гари,

поражение человека электрическим током, короткое замыкание, перегрев оборудования, разрушение корпуса устройства, утечка тонера, попадание жидкости внутрь оборудования, неисправность вентиляции, а также иные обстоятельства, создающие угрозу жизни и здоровью работников.

При появлении запаха гари, дыма, искрения или признаков короткого замыкания необходимо немедленно прекратить работу, по возможности отключить оборудование от электросети и сообщить о происшествии руководителю. Если отключение оборудования представляет опасность для работника, например при открытом искрении или угрозе поражения электрическим током, приближаться к устройству не следует. В этом случае необходимо обесточить помещение с помощью общего выключателя, если он расположен в безопасном месте, и вызвать ответственных лиц.

При возникновении пожара необходимо немедленно сообщить о пожаре в пожарную службу, оповестить находящихся поблизости людей и руководителя, а также приступить к тушению очага возгорания первичными средствами пожаротушения, если это не угрожает жизни и здоровью. Электрооборудование, находящееся под напряжением, нельзя тушить водой. Для тушения такой техники применяются углекислотные или порошковые огнетушители. При невозможности ликвидировать возгорание необходимо покинуть помещение согласно плану эвакуации [19].

Если произошло поражение человека электрическим током, необходимо немедленно освободить пострадавшего от действия электрического тока. Для этого следует отключить питание, выключить автоматический выключатель или вынуть вилку из розетки, соблюдая меры личной безопасности. Нельзя прикасаться к пострадавшему голыми руками, пока он находится под напряжением. После освобождения пострадавшего необходимо вызвать скорую медицинскую помощь и оказать первую помощь в пределах имеющихся навыков: проверить сознание, дыхание, пульс, при необходимости начать сердечно-легочную реанимацию.

При попадании жидкости на компьютер, клавиатуру, принтер или копировальный аппарат работу следует немедленно прекратить. Устройство необходимо отключить от электросети, не прикасаясь к мокрым участкам корпуса и проводов. Дальнейшая эксплуатация оборудования допускается только после проверки специалистом. Использование техники после попадания жидкости без осмотра может привести к короткому замыканию, поражению электрическим током или возгоранию.

При утечке тонера из картриджа или копировального аппарата следует избегать вдыхания порошка и попадания его на кожу, в глаза и на одежду. Нельзя сдувать тонер или использовать обычный бытовой пылесос, если он не предназначен для таких работ. Необходимо аккуратно ограничить распространение порошка, проветрить помещение и сообщить ответственному лицу. При попадании тонера на кожу его следует смыть прохладной водой с мылом. При попадании в глаза необходимо промыть их большим количеством воды и обратиться за медицинской помощью.

При аварийной ситуации работник обязан прекратить выполнение задания до полного устранения опасности. Возобновление работы допускается только после разрешения руководителя или специалиста, ответственного за исправность оборудования. Самовольное включение неисправной техники запрещается.

Таким образом, соблюдение требований охраны труда по окончании работы и в аварийных ситуациях позволяет предотвратить несчастные случаи, сохранить исправность оборудования и обеспечить безопасные условия труда. Каждый работник должен знать порядок отключения техники, правила поведения при неисправностях, способы вызова экстренных служб и расположение первичных средств пожаротушения.

## **5.2 Защита от воздействия производственных излучений**

В процессе трудовой деятельности человек может подвергаться воздействию различных видов производственных излучений. К ним относятся электромагнитные поля, инфракрасное, ультрафиолетовое, лазерное, ионизирующее и другие виды излучений, возникающие при эксплуатации производственного, офисного, медицинского, измерительного и технологического оборудования. В условиях работы с персональными компьютерами и множительной техникой основное значение имеют электромагнитные поля, оптическое излучение экранов, тепловое излучение отдельных узлов оборудования, а также световое воздействие, связанное с недостаточной или избыточной освещенностью рабочего места [20].

Современные персональные компьютеры, мониторы, принтеры и копировальные аппараты при нормальной эксплуатации не должны создавать опасных уровней излучения. Однако длительная работа за экраном, неправильная организация рабочего места, использование неисправного оборудования, недостаточное расстояние до монитора, блики на экране и нарушение режима труда и отдыха могут вызывать зрительное напряжение, головную боль,

утомление, снижение работоспособности, раздражительность и ухудшение самочувствия работника.

Одним из основных факторов при работе с ПК является зрительная нагрузка. Экран монитора является источником светового излучения, а также объектом постоянной концентрации внимания. При длительной работе глаза находятся в напряжении, особенно если изображение недостаточно четкое, шрифт слишком мелкий, экран расположен неправильно или освещение помещения не соответствует характеру выполняемой работы. Для снижения негативного воздействия необходимо правильно организовать рабочее место.

Монитор должен располагаться перед пользователем на расстоянии примерно 50–70 см от глаз. Верхняя граница экрана должна находиться на уровне глаз или немного ниже. Такое положение позволяет снизить напряжение мышц шеи и глаз. Экран необходимо устанавливать так, чтобы на нем не возникали блики от окон, светильников и отражающих поверхностей. При необходимости следует использовать жалюзи, шторы, рассеиватели света или изменить расположение рабочего стола [20].

Освещение рабочего места должно быть равномерным и достаточным. Слишком яркий свет, как и недостаточная освещенность, отрицательно влияет на зрение. Не допускается работа в темном помещении только при свете монитора, так как резкий контраст между экраном и окружающим пространством усиливает утомление глаз. Также нежелательно расположение монитора напротив окна или спиной к окну, если это приводит к появлению бликов. Наиболее рациональным считается боковое расположение источника естественного света.

Для защиты от воздействия электромагнитных полей необходимо использовать исправное оборудование, прошедшее проверку и соответствующее требованиям безопасности. Не допускается эксплуатация мониторов, системных блоков, блоков питания, принтеров и копировальных аппаратов с поврежденным корпусом, нарушенной изоляцией проводов, нестабильной работой или признаками перегрева. Оборудование должно быть подключено к исправной электрической сети с заземлением, если оно предусмотрено конструкцией устройства.

Не следует размещать рабочие места слишком близко друг к другу, особенно при большом количестве компьютерной техники в одном помещении. Системные блоки, источники бесперебойного питания, сетевые фильтры и кабели должны располагаться таким образом, чтобы не создавать опасности спотыкания, не мешать вентиляции и не приводить к перегреву оборудования.

Вентиляционные отверстия нельзя закрывать бумагами, тканью, мебелью или другими предметами.

Копировальные аппараты и лазерные принтеры в процессе работы могут выделять тепло, шум, запахи, а при нарушении условий эксплуатации — небольшое количество вредных веществ, связанных с использованием тонера и нагревательных элементов. Поэтому множительную технику целесообразно размещать в хорошо проветриваемых помещениях или зонах, где исключено постоянное нахождение работника непосредственно рядом с аппаратом во время интенсивной печати или копирования. При больших объемах копировальных работ следует предусматривать вентиляцию и регламентированные перерывы.

Отдельное внимание уделяется защите от ультрафиолетового и лазерного излучения. В обычных условиях эксплуатации лазерные принтеры и копировальные аппараты имеют закрытую конструкцию, поэтому лазерное излучение не выходит за пределы корпуса и не представляет опасности для пользователя. Однако разборка оборудования, вскрытие защитных крышек, самостоятельный ремонт или работа при снятых защитных элементах запрещаются. Такие действия могут привести не только к воздействию излучения, но и к поражению электрическим током или ожогам от нагретых деталей.

Средствами защиты от производственных излучений являются технические, организационные и индивидуальные меры. К техническим мерам относятся применение современного оборудования с низким уровнем излучения, исправное заземление, экранирование источников излучения, наличие защитных кожухов, правильная организация вентиляции и освещения. К организационным мерам относятся рациональное размещение рабочих мест, соблюдение режимов труда и отдыха, проведение инструктажей по охране труда, контроль исправности оборудования и своевременное техническое обслуживание [19, 20].

Большое значение имеет соблюдение режима труда и отдыха. При длительной работе за компьютером необходимо делать кратковременные перерывы, выполнять упражнения для глаз, менять положение тела, периодически вставать и разминаться. Это позволяет снизить зрительное и мышечное напряжение, предотвратить переутомление и повысить производительность труда. Во время перерывов рекомендуется отвести взгляд от экрана, посмотреть вдаль, выполнить несколько круговых движений глазами, моргнуть несколько раз для увлажнения глаз.

Для профилактики вредного воздействия излучений и других факторов офисной среды работник должен соблюдать правила личной гигиены. После

замены картриджей, контакта с бумагой, тонером или внутренними элементами множительной техники необходимо вымыть руки. Не рекомендуется принимать пищу непосредственно на рабочем месте, особенно рядом с принтером или копировальным аппаратом. Расходные материалы должны храниться в закрытой упаковке, вдали от источников тепла и влаги.

Контроль за безопасными условиями труда должен осуществляться ответственными лицами организации. Рабочие места должны соответствовать требованиям эргономики, освещенности, электробезопасности и пожарной безопасности. Работники должны проходить инструктажи по охране труда, знать порядок действий при аварийных ситуациях, уметь пользоваться первичными средствами пожаротушения и своевременно сообщать о неисправностях оборудования [19].

Таким образом, защита от воздействия производственных излучений при работе с ПК и множительной техникой обеспечивается комплексом мероприятий. К ним относятся правильная организация рабочего места, использование исправного оборудования, соблюдение безопасного расстояния до экрана, устранение бликов, обеспечение нормального освещения и вентиляции, соблюдение режима труда и отдыха, а также запрет на самостоятельное вскрытие и ремонт оборудования. Выполнение этих требований способствует сохранению здоровья работников, снижению утомляемости и созданию безопасных условий труда.

## ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была разработана информационная система «Чистый город», предназначенная для организации взаимодействия жителей, исполнителей и административного персонала при обработке обращений, связанных с проблемами городской среды. Разработанное решение позволяет упростить процесс подачи заявок, обеспечить их дальнейшее распределение между ответственными исполнителями и повысить удобство контроля за ходом выполнения работ.

На этапе анализа предметной области были рассмотрены основные проблемы, возникающие при обработке обращений граждан. Было установлено, что при отсутствии единой цифровой системы процесс фиксации, распределения и контроля заявок может занимать больше времени, а информация о выполнении работ не всегда хранится в удобном и структурированном виде. Это подтвердило актуальность разработки программного продукта, объединяющего мобильное приложение и административную веб-панель.

В процессе проектирования были определены основные роли пользователей системы: житель, исполнитель и администратор. Для каждой роли был предусмотрен собственный набор функций. Житель может создавать заявки, прикреплять фотографии, указывать местоположение проблемы и отслеживать статус обращения. Исполнитель получает назначенные заявки, принимает их в работу, выполняет необходимые действия и подтверждает результат. Администратор осуществляет контроль за обращениями, назначает исполнителей, управляет пользователями, организациями, новостями, купонами и просматривает статистическую информацию.

Практическая часть проекта включала разработку мобильного приложения для Android и серверной части с административной панелью. Мобильное приложение было реализовано на языке Java и взаимодействует с сервером через REST API. Серверная часть и административная панель разработаны с использованием Laravel. Для хранения основных данных системы применяется база данных MySQL. В системе предусмотрена работа с заявками, категориями, пользователями, ролями, организациями, статусами, фотографиями, координатами, баллами и элементами поощрения пользователей.

Особое внимание в проекте было уделено работе с геолокационными данными. При создании заявки пользователь может указать место проблемы автоматически с помощью геолокации устройства либо выбрать точку вручную на карте. Координаты используются для отображения заявок на карте, помогают

исполнителям быстрее находить место выполнения работ и создают основу для дальнейшего анализа проблемных участков городской территории.

Также в системе реализована административная часть, которая позволяет контролировать работу с обращениями. Через веб-панель администратор может просматривать список заявок, фильтровать их, назначать ответственных исполнителей, изменять статусы, просматривать фотографии и анализировать статистику. Это делает процесс обработки обращений более наглядным и управляемым.

Для проверки работоспособности системы было проведено тестирование ключевых сценариев. Были проверены функции авторизации, разграничения прав доступа, просмотра заявок, назначения исполнителей и изменения статусов обращений. Для административной части были разработаны функциональные тесты с использованием средств Laravel и PHPUnit. Результаты тестирования показали успешное выполнение проверяемых сценариев, что подтверждает корректность работы основных функций системы.

В результате выполнения дипломного проекта была создана работоспособная информационная система, которая решает поставленные задачи и может быть использована как основа для дальнейшего развития. В дальнейшем систему можно расширить за счёт добавления push-уведомлений, более развитой аналитики, автоматического построения маршрутов, расширенного механизма отчётности и интеграции с внешними городскими или коммунальными сервисами. Таким образом, разработанный программный продукт имеет практическую ценность и может способствовать повышению эффективности обработки обращений граждан, связанных с состоянием городской среды.



## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Android Developers. Android Studio User Guide [Электронный ресурс]. – Режим доступа: <https://developer.android.com/studio>. – Дата доступа: 10.06.2026.
2. Android Developers. Application fundamentals [Электронный ресурс]. – Режим доступа: <https://developer.android.com/guide/components/fundamentals>. – Дата доступа: 10.06.2026.
3. Leaflet Documentation [Электронный ресурс]. – Режим доступа: <https://leafletjs.com/reference.html>. – Дата доступа: 10.06.2026.
4. Oracle. Java Documentation [Электронный ресурс]. – Режим доступа: <https://docs.oracle.com/en/java/>. – Дата доступа: 10.06.2026.
5. PHP Documentation [Электронный ресурс]. – Режим доступа: <https://www.php.net/docs.php>. – Дата доступа: 10.06.2026.
6. Vite Documentation [Электронный ресурс]. – Режим доступа: <https://vite.dev/guide/>. – Дата доступа: 10.06.2026.
7. OpenStreetMap Wiki [Электронный ресурс]. – Режим доступа: [https://wiki.openstreetmap.org/wiki/Main\\_Page](https://wiki.openstreetmap.org/wiki/Main_Page). – Дата доступа: 10.06.2026.
8. Chart.js Documentation [Электронный ресурс]. – Режим доступа: <https://www.chartjs.org/docs/latest/>. – Дата доступа: 10.06.2026.
9. Laravel Documentation [Электронный ресурс]. – Режим доступа: <https://laravel.com/docs>. – Дата доступа: 10.06.2026.
10. Richardson, L., Amundsen, M., Ruby, S. RESTful Web APIs / L. Richardson, M. Amundsen, S. Ruby. – Sebastopol: O'Reilly Media, 2013. – 406 p.
11. MySQL 8.0 Reference Manual [Электронный ресурс]. – Режим доступа: <https://dev.mysql.com/doc/refman/8.0/en/>. – Дата доступа: 10.06.2026.
12. phpMyAdmin Documentation [Электронный ресурс]. – Режим доступа: <https://docs.phpmyadmin.net/>. – Дата доступа: 10.06.2026.
13. Гагарина, Л. Г., Кокорева, Е. В., Виснадул, Б. Д. Технология разработки программного обеспечения: учебное пособие / Л. Г. Гагарина, Е. В. Кокорева, Б. Д. Виснадул. – М.: ИД «ФОРУМ»: ИНФРА-М, 2021. – 280 с.
14. Кватрани, Т. Визуальное моделирование с помощью Rational Rose и UML / Т. Кватрани. – М.: Вильямс, 2019. – 195 с.
15. Плакса, Ю. Тестирование программного обеспечения. Системный подход / Ю. Плакса. – Минск: Беларуская навука, 2019. – 420 с.
16. PHPUnit Documentation [Электронный ресурс]. – Режим доступа: <https://docs.phpunit.de/>. – Дата доступа: 10.06.2026.

17. Суша, Г. З. Экономика предприятия: учебное пособие / Г. З. Суша. – Минск: Вышэйшая школа, 2020. – 480 с.
18. Полешук, И. И., Терешина, В. В. Ценообразование: учебник / И. И. Полешук, В. В. Терешина. – Минск: БГЭУ, 2020. – 220 с.
19. Андруш, В. Г., Ткачёва, П. Т., Яшин, К. Д. Охрана труда: учебник / В. Г. Андруш, П. Т. Ткачёва, К. Д. Яшин. – 3-е изд., перераб. и доп. – Минск: РИПО, 2022. – 350 с.
20. Пивоварчик, А. А. Охрана труда: учебно-методическое пособие / А. А. Пивоварчик. – Гродно: ГрГУ, 2022. – 440 с.

## ПРИЛОЖЕНИЕ А

### Код базы данных

#### Листинг А.1 - SQL-код создания основных таблиц базы данных

-- SQL-код создания основных таблиц базы данных приложения "Чистый город"

```
CREATE TABLE cities (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    region VARCHAR(255) NULL,
    active TINYINT(1) NOT NULL DEFAULT 1,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL
);

CREATE TABLE organizations (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    city_id BIGINT UNSIGNED NULL,
    district VARCHAR(255) NULL,
    address VARCHAR(255) NULL,
    lat DECIMAL(10,7) NULL,
    lng DECIMAL(10,7) NULL,
    contact_info VARCHAR(255) NULL,
    active TINYINT(1) NOT NULL DEFAULT 1,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX organizations_city_id_index (city_id),
    CONSTRAINT organizations_city_id_fk
        FOREIGN KEY (city_id) REFERENCES cities(id) ON DELETE SET NULL
);

CREATE TABLE users (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    email_verified_at TIMESTAMP NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(255) NOT NULL DEFAULT 'resident',
    organization_id BIGINT UNSIGNED NULL,
    avatar_path VARCHAR(255) NULL,
    banned_at TIMESTAMP NULL,
    ban_reason VARCHAR(255) NULL,
    banned_by_user_id BIGINT UNSIGNED NULL,
    points_balance INT NOT NULL DEFAULT 0,
    remember_token VARCHAR(100) NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX users_organization_id_index (organization_id),
    INDEX users_banned_by_user_id_index (banned_by_user_id),
    CONSTRAINT users_organization_id_fk
        FOREIGN KEY (organization_id) REFERENCES organizations(id) ON DELETE SET NULL,
    CONSTRAINT users_banned_by_user_id_fk
        FOREIGN KEY (banned_by_user_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE categories (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    icon VARCHAR(255) NULL,
    active TINYINT(1) NOT NULL DEFAULT 1,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL
);
```

```

CREATE TABLE tickets (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT UNSIGNED NOT NULL,
    category_id BIGINT UNSIGNED NOT NULL,
    assigned_org_id BIGINT UNSIGNED NULL,
    assigned_worker_id BIGINT UNSIGNED NULL,
    status VARCHAR(255) NOT NULL DEFAULT 'created',
    priority VARCHAR(255) NOT NULL DEFAULT 'normal',
    available_to_residents TINYINT(1) NOT NULL DEFAULT 0,
    lat DECIMAL(10,7) NOT NULL,
    lng DECIMAL(10,7) NOT NULL,
    address_text VARCHAR(255) NULL,
    description VARCHAR(200) NULL,
    closed_at TIMESTAMP NULL,
    deleted_by_user_id BIGINT UNSIGNED NULL,
    deleted_at TIMESTAMP NULL,
    delete_reason VARCHAR(255) NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX tickets_user_id_index (user_id),
    INDEX tickets_category_id_index (category_id),
    INDEX tickets_assigned_org_id_index (assigned_org_id),
    INDEX tickets_assigned_worker_id_index (assigned_worker_id),
    INDEX tickets_status_index (status),
    INDEX tickets_priority_index (priority),
    INDEX tickets_deleted_by_user_id_index (deleted_by_user_id),
    INDEX tickets_deleted_at_index (deleted_at),
    CONSTRAINT tickets_user_id_fk
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT tickets_category_id_fk
        FOREIGN KEY (category_id) REFERENCES categories(id),
    CONSTRAINT tickets_assigned_org_id_fk
        FOREIGN KEY (assigned_org_id) REFERENCES organizations(id) ON DELETE SET NULL,
    CONSTRAINT tickets_assigned_worker_id_fk
        FOREIGN KEY (assigned_worker_id) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT tickets_deleted_by_user_id_fk
        FOREIGN KEY (deleted_by_user_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE ticket_photos (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ticket_id BIGINT UNSIGNED NOT NULL,
    type VARCHAR(255) NOT NULL,
    path VARCHAR(255) NOT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX ticket_photos_ticket_id_index (ticket_id),
    INDEX ticket_photos_type_index (type),
    CONSTRAINT ticket_photos_ticket_id_fk
        FOREIGN KEY (ticket_id) REFERENCES tickets(id) ON DELETE CASCADE
);

CREATE TABLE ticket_status_histories (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ticket_id BIGINT UNSIGNED NOT NULL,
    old_status VARCHAR(255) NULL,
    new_status VARCHAR(255) NOT NULL,
    changed_by_user_id BIGINT UNSIGNED NULL,
    comment VARCHAR(255) NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX ticket_status_histories_ticket_id_index (ticket_id),
    INDEX ticket_status_histories_changed_by_user_id_index (changed_by_user_id),
    INDEX ticket_status_histories_new_status_index (new_status),
    CONSTRAINT ticket_status_histories_ticket_id_fk
        FOREIGN KEY (ticket_id) REFERENCES tickets(id) ON DELETE CASCADE,
    CONSTRAINT ticket_status_histories_changed_by_user_id_fk
        FOREIGN KEY (changed_by_user_id) REFERENCES users(id) ON DELETE SET NULL
);

```

```

CREATE TABLE api_tokens (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT UNSIGNED NOT NULL,
    name VARCHAR(255) NOT NULL DEFAULT 'api-token',
    token_hash VARCHAR(64) NOT NULL UNIQUE,
    last_used_at TIMESTAMP NULL,
    expires_at TIMESTAMP NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX api_tokens_user_id_index (user_id),
    CONSTRAINT api_tokens_user_id_fk
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE worker_registration_codes (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(32) NOT NULL UNIQUE,
    issued_to VARCHAR(120) NULL,
    organization_id BIGINT UNSIGNED NOT NULL,
    created_by_user_id BIGINT UNSIGNED NULL,
    used_by_user_id BIGINT UNSIGNED NULL,
    used_at TIMESTAMP NULL,
    max_uses INT UNSIGNED NOT NULL DEFAULT 1,
    used_count INT UNSIGNED NOT NULL DEFAULT 0,
    expires_at TIMESTAMP NULL,
    active TINYINT(1) NOT NULL DEFAULT 1,
    revoked_at TIMESTAMP NULL,
    revoked_by_user_id BIGINT UNSIGNED NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX worker_registration_codes_organization_id_index (organization_id),
    INDEX worker_registration_codes_created_by_user_id_index (created_by_user_id),
    INDEX worker_registration_codes_used_by_user_id_index (used_by_user_id),
    INDEX worker_registration_codes_revoked_by_user_id_index (revoked_by_user_id),
    INDEX worker_registration_codes_active_index (active),
    INDEX worker_registration_codes_expires_at_index (expires_at),
    CONSTRAINT worker_registration_codes_organization_id_fk
        FOREIGN KEY (organization_id) REFERENCES organizations(id) ON DELETE CASCADE,
    CONSTRAINT worker_registration_codes_created_by_user_id_fk
        FOREIGN KEY (created_by_user_id) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT worker_registration_codes_used_by_user_id_fk
        FOREIGN KEY (used_by_user_id) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT worker_registration_codes_revoked_by_user_id_fk
        FOREIGN KEY (revoked_by_user_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE organization_user_blocks (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    organization_id BIGINT UNSIGNED NOT NULL,
    user_id BIGINT UNSIGNED NOT NULL,
    created_by_user_id BIGINT UNSIGNED NULL,
    reason VARCHAR(255) NULL,
    active TINYINT(1) NOT NULL DEFAULT 1,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX organization_user_blocks_organization_id_index (organization_id),
    INDEX organization_user_blocks_user_id_index (user_id),
    INDEX organization_user_blocks_created_by_user_id_index (created_by_user_id),
    INDEX organization_user_blocks_active_index (active),
    CONSTRAINT organization_user_blocks_organization_id_fk
        FOREIGN KEY (organization_id) REFERENCES organizations(id) ON DELETE CASCADE,
    CONSTRAINT organization_user_blocks_user_id_fk
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT organization_user_blocks_created_by_user_id_fk
        FOREIGN KEY (created_by_user_id) REFERENCES users(id) ON DELETE SET NULL
);

```

```

CREATE TABLE ticket_hides (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ticket_id BIGINT UNSIGNED NOT NULL,
    organization_id BIGINT UNSIGNED NULL,
    hidden_by_user_id BIGINT UNSIGNED NULL,
    reason VARCHAR(255) NULL,
    active TINYINT(1) NOT NULL DEFAULT 1,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX ticket_hides_ticket_id_index (ticket_id),
    INDEX ticket_hides_organization_id_index (organization_id),
    INDEX ticket_hides_hidden_by_user_id_index (hidden_by_user_id),
    INDEX ticket_hides_active_index (active),
    CONSTRAINT ticket_hides_ticket_id_fk
        FOREIGN KEY (ticket_id) REFERENCES tickets(id) ON DELETE CASCADE,
    CONSTRAINT ticket_hides_organization_id_fk
        FOREIGN KEY (organization_id) REFERENCES organizations(id) ON DELETE SET NULL,
    CONSTRAINT ticket_hides_hidden_by_user_id_fk
        FOREIGN KEY (hidden_by_user_id) REFERENCES users(id) ON DELETE SET NULL
);

```

```

CREATE TABLE organization_complaints (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    organization_id BIGINT UNSIGNED NOT NULL,
    created_by_user_id BIGINT UNSIGNED NOT NULL,
    target_user_id BIGINT UNSIGNED NULL,
    ticket_id BIGINT UNSIGNED NULL,
    type VARCHAR(40) NOT NULL DEFAULT 'other',
    status VARCHAR(30) NOT NULL DEFAULT 'pending',
    title VARCHAR(160) NOT NULL,
    description TEXT NULL,
    reviewed_by_user_id BIGINT UNSIGNED NULL,
    reviewed_at TIMESTAMP NULL,
    resolution TEXT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX organization_complaints_organization_id_index (organization_id),
    INDEX organization_complaints_created_by_user_id_index (created_by_user_id),
    INDEX organization_complaints_target_user_id_index (target_user_id),
    INDEX organization_complaints_ticket_id_index (ticket_id),
    INDEX organization_complaints_type_index (type),
    INDEX organization_complaints_status_index (status),
    INDEX organization_complaints_reviewed_by_user_id_index (reviewed_by_user_id),
    CONSTRAINT organization_complaints_organization_id_fk
        FOREIGN KEY (organization_id) REFERENCES organizations(id) ON DELETE CASCADE,
    CONSTRAINT organization_complaints_created_by_user_id_fk
        FOREIGN KEY (created_by_user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT organization_complaints_target_user_id_fk
        FOREIGN KEY (target_user_id) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT organization_complaints_ticket_id_fk
        FOREIGN KEY (ticket_id) REFERENCES tickets(id) ON DELETE SET NULL,
    CONSTRAINT organization_complaints_reviewed_by_user_id_fk
        FOREIGN KEY (reviewed_by_user_id) REFERENCES users(id) ON DELETE SET NULL
);

```

```

CREATE TABLE ticket_claim_requests (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ticket_id BIGINT UNSIGNED NOT NULL,
    worker_id BIGINT UNSIGNED NOT NULL,
    organization_id BIGINT UNSIGNED NOT NULL,
    status VARCHAR(30) NOT NULL DEFAULT 'pending',
    comment VARCHAR(255) NULL,
    reviewed_by_user_id BIGINT UNSIGNED NULL,
    reviewed_at TIMESTAMP NULL,
    resolution TEXT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX ticket_claim_requests_ticket_id_index (ticket_id),
    INDEX ticket_claim_requests_worker_id_index (worker_id),

```

```

INDEX ticket_claim_requests_organization_id_index (organization_id),
INDEX ticket_claim_requests_status_index (status),
CONSTRAINT ticket_claim_requests_ticket_id_fk
    FOREIGN KEY (ticket_id) REFERENCES tickets(id) ON DELETE CASCADE,
CONSTRAINT ticket_claim_requests_worker_id_fk
    FOREIGN KEY (worker_id) REFERENCES users(id) ON DELETE CASCADE,
CONSTRAINT ticket_claim_requests_organization_id_fk
    FOREIGN KEY (organization_id) REFERENCES organizations(id) ON DELETE CASCADE,
CONSTRAINT ticket_claim_requests_reviewed_by_user_id_fk
    FOREIGN KEY (reviewed_by_user_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE news (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    body TEXT NULL,
    published_date DATE NOT NULL,
    active TINYINT(1) NOT NULL DEFAULT 1,
    created_by_user_id BIGINT UNSIGNED NULL,
    organization_id BIGINT UNSIGNED NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX news_created_by_user_id_index (created_by_user_id),
    INDEX news_organization_id_index (organization_id),
    CONSTRAINT news_created_by_user_id_fk
        FOREIGN KEY (created_by_user_id) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT news_organization_id_fk
        FOREIGN KEY (organization_id) REFERENCES organizations(id) ON DELETE SET NULL
);

CREATE TABLE news_photos (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    news_id BIGINT UNSIGNED NOT NULL,
    path VARCHAR(255) NOT NULL,
    sort_order INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    CONSTRAINT news_photos_news_id_fk
        FOREIGN KEY (news_id) REFERENCES news(id) ON DELETE CASCADE
);

CREATE TABLE rewards (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT NULL,
    photo_path VARCHAR(255) NULL,
    points_required INT NOT NULL DEFAULT 0,
    code VARCHAR(255) NULL,
    valid_from DATE NULL,
    valid_to DATE NULL,
    active TINYINT(1) NOT NULL DEFAULT 1,
    created_by_user_id BIGINT UNSIGNED NULL,
    organization_id BIGINT UNSIGNED NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    INDEX rewards_created_by_user_id_index (created_by_user_id),
    INDEX rewards_organization_id_index (organization_id),
    CONSTRAINT rewards_created_by_user_id_fk
        FOREIGN KEY (created_by_user_id) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT rewards_organization_id_fk
        FOREIGN KEY (organization_id) REFERENCES organizations(id) ON DELETE SET NULL
);

CREATE TABLE points_transactions (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT UNSIGNED NOT NULL,
    amount INT NOT NULL,
    balance_after INT NOT NULL,

```

```

reason VARCHAR(255) NULL,
admin_id BIGINT UNSIGNED NULL,
created_at TIMESTAMP NULL,
updated_at TIMESTAMP NULL,
INDEX points_transactions_user_id_index (user_id),
INDEX points_transactions_admin_id_index (admin_id),
CONSTRAINT points_transactions_user_id_fk
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
CONSTRAINT points_transactions_admin_id_fk
    FOREIGN KEY (admin_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE system_settings (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `key` VARCHAR(255) NOT NULL UNIQUE,
    value TEXT NULL,
    description VARCHAR(255) NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL
);

```